

โฟลเดอร์ artifacts/

-ไฟล์ ablation_table.md

```
| Variant | RECALL@1 | RECALL@3 | RECALL@5 | MRR | Latency (ms) |  
|---|---|---|---|---|---|  
| `dense` | 0.9000 | 1.0000 | 1.0000 | 0.9417 | 45.84 |  
| `bm25` | 0.9500 | 0.9500 | 1.0000 | 0.9625 | 70.19 |  
| `hybrid` | 0.9500 | 1.0000 | 1.0000 | 0.9750 | 83.54 |  
| `hybrid_rerank` | 1.0000 | 1.0000 | 1.0000 | 1.0000 | 17848.89 |
```

โฟลเดอร์ data/

-โฟลเดอร์ documents/

-โฟลเดอร์ pdf/

โฟลเดอร์ scripts

-ไฟล์ build_index.py

```
import sys; sys.path.insert(0, ".")  
from src.chunking import ThaiLegalChunker, chunks_to_dicts  
from src.indexer import Indexer  
  
chunker = ThaiLegalChunker()  
chunks = chunker.chunk_corpus()  
  
sizes = [c.n_tokens for c in chunks]  
print(f"Total chunks: {len(chunks)} | avg tokens:  
{sum(sizes)/len(sizes):.0f} "  
      f"| min: {min(sizes)} | max: {max(sizes)}")  
  
Indexer().build(chunks_to_dicts(chunks))
```

-ไฟล์ run_ablation.py

```
"""Ablation Study: เปรียบเทียบ 4 Retrieval Variants บนชุดข้อมูลทดสอบ  
- Variant 1: dense (intfloat/multilingual-e5-base)  
- Variant 2: bm25 (Sparse lexical search ด้วย PyThaiNLP)  
- Variant 3: hybrid (Dense + BM25 รวมด้วย RRF)  
- Variant 4: hybrid_rerank (Hybrid + BAAI/bge-reranker-v2-m3)
```

```

วัดผลทั้ง Retrieval Metrics (Recall, Hit, nDCG, MRR), Latency, และ
Statistical Significance
"""
from __future__ import annotations

import json
import sys
import time
from pathlib import Path

# ปรับ path ให้มองเห็น root directory เสมอ
ROOT_DIR = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT_DIR))

import config
from src.retrieval import Retriever
from src.metrics import (
    gold_docs_of,
    retrieved_docs_of,
    recall_at_k,
    hit_at_k,
    mrr,
    ndcg_at_k,
    mean,
    bootstrap_ci,
    paired_bootstrap_p,
)

VARIANTS = ["dense", "bm25", "hybrid", "hybrid_rerank"]

def main():
    # 1. จัดการ Path และโหลด Testset อย่างปลอดภัย
    testset_path = getattr(config, "TESTSET_PATH", ROOT_DIR / "data" /
"testset.json")
    with open(testset_path, encoding="utf-8") as f:
        raw_testset = json.load(f)

    if isinstance(raw_testset, dict):

```

```

        testset = raw_testset.get("questions") or raw_testset.get("data")
    or []
    else:
        testset = raw_testset

    # กรองเฉพาะข้อที่มี ground truth
    items = [t for t in testset if not t.get("adversarial")]
    items = [t for t in items if gold_docs_of(t)]

    if not items:
        raise SystemExit("❌ ไม่พบข้อคำถามที่มี gold doc - โปรดตรวจสอบ
testset.json")

    print(f"✓ พร้อมประเมิน {len(items)} คำถาม × {len(VARIANTS)} variants\n")

    # 2. ดึงค่าคอนฟิกพร้อม Fallback ป้องกันตัวแปรหาย
    eval_k_values = getattr(config, "EVAL_K_VALUES", [1, 3, 5])
    max_k = getattr(config, "MAX_EVAL_K", max(eval_k_values))
    rerank_pool = getattr(config, "RERANK_CANDIDATES", getattr(config,
"TOP_K_RETRIEVE", 30))
    n_boot = getattr(config, "N_BOOTSTRAP", 2000)
    seed = getattr(config, "RANDOM_SEED", 42)

    artifacts_dir = Path(getattr(config, "ARTIFACTS_DIR", getattr(config,
"INDEX_DIR", ROOT_DIR / "artifacts")))
    artifacts_dir.mkdir(exist_ok=True, parents=True)

    r = Retriever(index_dir=artifacts_dir)
    per_q: dict[str, dict[str, list]] = {v: {} for v in VARIANTS}
    diagnostics = {v: {"latency": [], "rank_shift": [], "n_cand": []} for
v in VARIANTS}

    # 3. รันประเมินผลแต่ละคำถาม
    for n, item in enumerate(items, 1):
        q = item.get("question") or item.get("query")
        gold = gold_docs_of(item)

        for v in VARIANTS:
            t0 = time.perf_counter()
            hits, dbg = r.retrieve(

```

```

        q,
        mode=v,
        k=max_k,
        retrieve_k=rerank_pool,
        return_debug=True,
    )
    elapsed = time.perf_counter() - t0

    diagnostics[v]["latency"].append(elapsed)
    diagnostics[v]["rank_shift"].append(dbg.get("rank_shift", 0))
    diagnostics[v]["n_cand"].append(dbg.get("n_candidates",
dbg.get("n_returned", 0)))

    docs = retrieved_docs_of(hits)
    d = per_q[v]

    for k_val in eval_k_values:
        d.setdefault(f"recall@{k_val}",
[]) .append(recall_at_k(docs, gold, k_val))
        d.setdefault(f"hit@{k_val}", []).append(hit_at_k(docs,
gold, k_val))
        d.setdefault(f"ndcg@{k_val}", []).append(ndcg_at_k(docs,
gold, k_val))

    d.setdefault("mrr", []).append(mrr(docs, gold, max_k))

    print(f"    ความคืบหน้า: [{n}/{len(items)}] คำถามเรียบร้อย", end="\r")

print("\nประมวลผลเสร็จสิ้น กำลังสรุปสถิติ...")

# 4. รวมผลคะแนนและสถิติ Bootstrap CI
results = {}
for v in VARIANTS:
    agg = {}
    for m, vals in per_q[v].items():
        lo, hi = bootstrap_ci(vals, n_boot=n_boot, seed=seed)
        agg[m] = {"mean": round(mean(vals), 4), "ci95": [round(lo, 4),
round(hi, 4)]}
    agg["latency_ms_avg"] = round(1000 *
mean(diagnostics[v]["latency"]), 2)

```

```

        agg["avg_candidates"] = round(mean(diagnostics[v]["n_cand"]), 1)
        if v == "hybrid_rerank":
            agg["avg_rank_shift"] =
round(mean(diagnostics[v]["rank_shift"]), 2)
        results[v] = agg

# 5. คำนวณความต่างอย่างมีนัยสำคัญทางสถิติ (เทียบกับ Dense-only)
sig = {}
target_sig_metrics = [f"recall@{k}" for k in eval_k_values if
f"recall@{k}" in per_q["dense"]]
target_sig_metrics.append("mrr")

for v in VARIANTS:
    if v == "dense":
        continue
    sig[v] = {
        m: round(paired_bootstrap_p(per_q[v][m], per_q["dense"][m],
n_boot=n_boot, seed=seed), 4)
        for m in target_sig_metrics
        if m in per_q[v] and m in per_q["dense"]
    }

# 6. บันทึกผลลัพธ์ JSON
out = {
    "n_questions": len(items),
    "k_values": eval_k_values,
    "config": {
        "TOP_K_RETRIEVE": getattr(config, "TOP_K_RETRIEVE", 30),
        "RERANK_CANDIDATES": rerank_pool,
        "TOP_K_FINAL": getattr(config, "TOP_K_FINAL", 5),
        "RRF_K": getattr(config, "RRF_K", 60),
    },
    "results": results,
    "p_values_vs_dense": sig,
    "per_question": per_q,
}

out_json = artifacts_dir / "ablation_results.json"
out_json.write_text(json.dumps(out, ensure_ascii=False, indent=2),
encoding="utf-8")

```

```

# 7. จัดทำและบันทึกตาราง Markdown สำหรับใช้ในสไลด์พรีเซนต์
disp_cols = [f"recall@{k}" for k in eval_k_values] + ["mrr"]
header = ["| Variant |"] + [f" {c.upper()} |" for c in disp_cols] + ["
Latency (ms) |"]
sep = ["|---"] * (len(disp_cols) + 2) + ["|"]

md_lines = ["".join(header), "".join(sep)]
for v in VARIANTS:
    row_str = f"| `{v}` |"
    for c in disp_cols:
        val = results[v].get(c, {}).get("mean", 0.0)
        row_str += f" {val:.4f} |"
    row_str += f" {results[v]['latency_ms_avg']} |"
    md_lines.append(row_str)

md_table = "\n".join(md_lines)
(artifacts_dir / "ablation_table.md").write_text(md_table,
encoding="utf-8")

print("\n" + "=" * 60)
print(md_table)
print("=" * 60)

rs = results["hybrid_rerank"].get("avg_rank_shift", 0)
print(f"🔄 Cross-Encoder Reranker สลับอันดับเฉลี่ย: {rs} ตำแหน่ง/คำถาม")
print(f"📁 บันทึกผลลัพธ์ที่: {out_json}")
print(f"📄 บันทึกตาราง Markdown ที่: {artifacts_dir /
'ablation_table.md'}\n")

if __name__ == "__main__":
    main()

```

- ไฟล์ run_eval.py

"""Evaluation Script: รันการประเมิน Full RAG Pipeline บนชุดข้อมูลทดสอบ

ประเมินผลครอบคลุมทั้ง:

1. BERTScore (xlm-roberta-large)
2. LLM-as-Judge (Faithfulness & Relevance พร้อมแจกแจง Distribution)
3. Abstention Analysis (วัดความแม่นยำในการปฏิเสธคำตอบเพื่อลด Hallucination)
4. Failure Analysis Extraction (ส่งออกเคสที่มีปัญหาลง failure_cases.json อัตโนมัติ)

"""

```
from __future__ import annotations
```

```
import json
```

```
import sys
```

```
from pathlib import Path
```

```
from tqdm import tqdm
```

```
# ปรับ Root Directory ให้ค้นหาโมดูลในโปรเจกต์เจอเสมอ
```

```
ROOT_DIR = Path(__file__).resolve().parent.parent
```

```
sys.path.insert(0, str(ROOT_DIR))
```

```
import config
```

```
from src.pipeline import RAGPipeline
```

```
from src.generation import is_abstention, build_context
```

```
from src.evaluation import llm_judge, compute_bertscore
```

```
from src.metrics import mean, bootstrap_ci
```

```
def main():
```

```
    # 1. โหลด Testset อย่างปลอดภัย
```

```
    testset_path = getattr(config, "TESTSET_PATH", ROOT_DIR / "data" /  
"testset.json")
```

```
    with open(testset_path, encoding="utf-8") as f:
```

```
        raw_testset = json.load(f)
```

```
    if isinstance(raw_testset, dict):
```

```
        testset = raw_testset.get("questions") or raw_testset.get("data")
```

```
    or []
```

```
    else:
```

```
        testset = raw_testset
```

```
    print(f"✓ เริ่มรับการประเมินผล Full Pipeline บนคำถาม {len(testset)} ข้อ  
...\n")
```

```

pipeline = RAGPipeline(mode="hybrid_rerank")
rows = []

# 2. วนลูปประเมินผลคำถามทีละข้อ
for item in tqdm(testset, desc="Evaluating Pipeline"):
    q = item.get("question") or item.get("query")
    ref = item.get("reference_answer") or item.get("ground_truth") or ""

    adv = bool(item.get("adversarial", False))

    # รัน Pipeline (เรียก answer จาก src/pipeline.py)
    result = pipeline.answer(q)
    ans = result.get("answer", "")
    contexts = result.get("contexts", [])

    # สร้าง Context Text ส่งให้ LLM-as-Judge
    context_text = build_context(contexts)
    judge = llm_judge(q, context_text, ans)

    # ตรวจสอบการปฏิเสธคำตอบ
    abstained = is_abstention(ans)

    rows.append({
        "question": q,
        "answer": ans,
        "reference": ref,
        "adversarial": adv,
        "abstained": abstained,
        "retrieved_docs": [c.get("doc_id") or c.get("file") for c in
contexts],
        "judge": judge,
    })

normal = [r for r in rows if not r["adversarial"]]
adv_rows = [r for r in rows if r["adversarial"]]

# 3. คำนวณ BERTScore เฉพาะข้อที่มี Ground Truth อ้างอิง
scored = [r for r in normal if r["reference"].strip()]
if scored:

```

```

print("\nกำลังคำนวณ BERTScore (xlm-roberta-large)...")
bs = compute_bertscore(
    [r["answer"] for r in scored],
    [r["reference"] for r in scored]
)
else:
    bs = {"precision": 0.0, "recall": 0.0, "f1": 0.0}

# 4. รวมคะแนนฝั่ง LLM-as-Judge
def get_judge_scores(key: str, subset: list[dict]) -> list[float]:
    return [r["judge"][key] for r in subset if r["judge"].get(key) is
not None]

f_all = get_judge_scores("faithfulness", rows)
r_all = get_judge_scores("relevance", rows)

n_boot = getattr(config, "N_BOOTSTRAP", 2000)
seed = getattr(config, "RANDOM_SEED", 42)
f_ci = bootstrap_ci(f_all, n_boot=n_boot, seed=seed) if f_all else
[0.0, 0.0]

summary = {
    "n_total": len(rows),
    "n_answerable": len(normal),
    "n_adversarial": len(adv_rows),
    "bertscore": bs,
    "llm_judge": {
        "faithfulness_avg": round(mean(f_all), 3),
        "faithfulness_ci95": [round(x, 3) for x in f_ci],
        "faithfulness_min": min(f_all) if f_all else None,
        "faithfulness_distribution": {str(s): f_all.count(s) for s in
(1, 2, 3, 4, 5)},
        "relevance_avg": round(mean(r_all), 3),
        "relevance_distribution": {str(s): r_all.count(s) for s in (1,
2, 3, 4, 5)},
        "n_judge_failed": sum(1 for r in rows if not
r["judge"].get("ok")),
    },
    "abstention": {

```

```

        "correct_abstention_rate": round(mean([1.0 if r["abstained"]
else 0.0 for r in adv_rows]), 3) if adv_rows else None,
        "false_abstention_rate": round(mean([1.0 if r["abstained"]
else 0.0 for r in normal]), 3) if normal else None,
    },
}

# 5. วิเคราะห์และดึง Failure Cases อัตโนมัติสำหรับ Slide 9
failures = []
for r in rows:
    j = r["judge"]
    why = []
    if r["adversarial"] and not r["abstained"]:
        why.append("HALLUCINATION: คำถามหลอกที่ควรปฏิเสธแต่กลับตอบ")
    if not r["adversarial"] and r["abstained"]:
        why.append("OVER-REFUSAL: ตอบได้แต่ปฏิเสธ (Retriever ดึงเอกสารไม่เจอ
)")
    if (j.get("faithfulness") or 5) <= 3:
        why.append(f"LOW FAITHFULNESS ({j.get('faithfulness')})")
    if (j.get("relevance") or 5) <= 3:
        why.append(f"LOW RELEVANCE ({j.get('relevance')})")
    if j.get("unsupported_claims"):
        why.append(f"UNSUPPORTED CLAIMS
({len(j['unsupported_claims'])} ประเด็น)")

    if why:
        failures.append(**r, "failure_types": why)

# 6. บันทึกผลลัพธ์ลงไฟล์
artifacts_dir = Path(getattr(config, "ARTIFACTS_DIR", getattr(config,
"INDEX_DIR", ROOT_DIR / "artifacts")))
artifacts_dir.mkdir(exist_ok=True, parents=True)

out_eval = artifacts_dir / "eval_results.json"
out_failures = artifacts_dir / "failure_cases.json"

out_eval.write_text(
    json.dumps(**summary, "per_question": rows), ensure_ascii=False,
indent=2),
    encoding="utf-8"

```

```

    )
    out_failures.write_text(
        json.dumps(failures, ensure_ascii=False, indent=2),
        encoding="utf-8"
    )

# 7. แสดงผลสรุปออกทางหน้าจอ
print("\n" + "=" * 60)
print("ผลการประเมินระบบ RAG ประจำปีโครงการ (Evaluation Summary):")
print(json.dumps(summary, ensure_ascii=False, indent=2))
print("=" * 60)
print(f"📉 พบจุดบกพร่อง (Failures) ทั้งหมด: {len(failures)}/{len(rows)} ข้อ")
print(f"📁 บันทึกผลการประเมินที่: {out_eval}")
print(f"📄 นำเคสในไฟล์นี้ไปใส่ Slide 9: {out_failures}\n")

if __name__ == "__main__":
    main()

```

-ไฟล์ smoke_test.py

```

"""Smoke Test & System Sanity Check
สคริปต์ตรวจสอบความสมบูรณ์ของระบบแบบครบวงจร (Configuration, Metrics, Testset, และ
Live Retriever)
ก่อนเริ่มรันสคริปต์ประเมินผลจริง (scripts/run_ablation.py และ scripts/run_eval.py)
"""
from __future__ import annotations

import json
import sys
from pathlib import Path

ROOT_DIR = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT_DIR))

import config
from src.metrics import norm_doc, gold_docs_of, recall_at_k, hit_at_k,
mrr, ndcg_at_k

```

```

PASS, FAIL = "✅", "❌"
errors = []

def check(name: str, cond: bool):
    print(f"{PASS if cond else FAIL} {name}")
    if not cond:
        errors.append(name)

print("— 1. Configuration Sanity —")
top_k_final = getattr(config, "TOP_K_FINAL", 5)
top_k_retrieve = getattr(config, "TOP_K_RETRIEVE", 30)
rerank_cands = getattr(config, "RERANK_CANDIDATES", top_k_retrieve)
eval_k_vals = getattr(config, "EVAL_K_VALUES", [1, 3, 5])
max_eval_k = getattr(config, "MAX_EVAL_K", max(eval_k_vals))

check("Candidate Pool > TOP_K_FINAL", rerank_cands > top_k_final)
check("TOP_K_RETRIEVE >= MAX_EVAL_K", top_k_retrieve >= max_eval_k)
check("Endpoint URL Configured", bool(getattr(config, "LLM_BASE_URL", "")))
check("Model Configured", bool(getattr(config, "LLM_MODEL", "")))

print("\n— 2. Information Retrieval Metrics —")
check("norm_doc ตัดนามสกุลและพาร", norm_doc("data/docs/A.TXT") == "a")
check("gold จาก source_doc", gold_docs_of({"source_doc": "civil.txt"}) == {"civil"})
check("gold จาก relevant_docs", gold_docs_of({"relevant_docs": ["a.txt", "b"]}) == {"a", "b"})

sample_ret, sample_gold = ["x", "a", "y", "b", "z"], {"a", "b"}
check("recall@5 = 1.0", recall_at_k(sample_ret, sample_gold, 5) == 1.0)
check("recall@1 = 0.0", recall_at_k(sample_ret, sample_gold, 1) == 0.0)
check("hit@2 = 1.0", hit_at_k(sample_ret, sample_gold, 2) == 1.0)
check("mrr = 0.5", abs(mrr(sample_ret, sample_gold, 5) - 0.5) < 1e-9)
check("ndcg@5 อยู่ใน [0, 1]", 0 <= ndcg_at_k(sample_ret, sample_gold, 5) <= 1)
check("gold วาง คำนวณ 0.0 ปลอดภัย", recall_at_k(sample_ret, set(), 5) == 0.0)

```

```

print("\n— 3. Testset & Corpus Sync —")
testset_path = getattr(config, "TESTSET_PATH", ROOT_DIR / "data" /
"testset.json")
artifacts_dir = Path(getattr(config, "ARTIFACTS_DIR", getattr(config,
"INDEX_DIR", ROOT_DIR / "artifacts")))

with open(testset_path, encoding="utf-8") as f:
    ts = json.load(f)
ts = ts.get("questions", ts) if isinstance(ts, dict) else ts

adv = [t for t in ts if t.get("adversarial")]
normal = [t for t in ts if not t.get("adversarial")]

check(f"มี Adversarial Questions >= 5 ข้อ (พบ {len(adv)} ข้อ)", len(adv) >=
5)
check("คำถามปกติทุกข้อมี Gold Docs ครอบคลุม", all(gold_docs_of(t) for t in
normal))

with open(artifacts_dir / "chunks.json", encoding="utf-8") as f:
    chunk_data = json.load(f)
chunk_docs = {norm_doc(c.get("doc_id") or c.get("file") or
str(c.get("chunk_id", "")).rsplit("::", 1)[0]) for c in chunk_data}

orphan = [t.get("id") for t in normal if not (gold_docs_of(t) &
chunk_docs)]
check(f"เอกสารอ้างอิงทุกข้อมีอยู่ใน Corpus (Orphans: {orphan[:3]})", not orphan)

print("\n— 4. Live Retriever Tests —")
from src.retrieval import Retriever

r = Retriever(index_dir=artifacts_dir)
sample_query = normal[0].get("question") or normal[0].get("query")

for m in ("dense", "bm25", "hybrid"):
    hits = r.retrieve(sample_query, mode=m, k=5)
    check(f"Retriever mode='{m}' คืนค่า 5 Chunks", len(hits) == 5)

hits, dbg = r.retrieve(
    sample_query,

```

```

mode="hybrid_rerank",
k=5,
retrieve_k=rerank_cands,
return_debug=True,
)
check(f"Candidate ป้อนเข้า Reranker > 5 (ได้ {dbg.get('n_candidates',
len(hits))})", dbg.get("n_candidates", 0) > 5)
check("ผลลัพธ์สุดท้าย hybrid_rerank = 5", len(hits) == 5)
check("ทุก Chunk มี rerank_score", all("rerank_score" in h for h in hits))
check(
    "คะแนน rerank_score เรียงจากมากไปน้อย",
    all(hits[i]["rerank_score"] >= hits[i + 1]["rerank_score"] for i in
range(len(hits) - 1)),
)

check("Query วางเปล่าไม่ Crash", r.retrieve(" ", mode="hybrid") == [])

try:
    r.retrieve(sample_query, mode="invalid_mode")
    check("Mode ผิดต้อง Raise ValueError", False)
except ValueError:
    check("Mode ผิดต้อง Raise ValueError", True)

print("\n" + ("=" * 50))
if not errors:
    print(f"{PASS} ผ่านการทดสอบทั้งหมด ระบบพร้อมรัน run_ablation.py และ
run_eval.py")
else:
    print(f"{FAIL} พบปัญหา {len(errors)} รายการที่ต้องแก้ไขก่อนรันจริง: {errors}")

sys.exit(1 if errors else 0)

```

โฟลเดอร์ src/

-ไฟล์ __init__.py

(วางเปล่า ไม่มีอะไรเลย ดังนั้นจึงไม่แน่ใจว่าควรลบไฟล์นี้ทิ้งไหม)

-ไฟล์ [chunking.py](#)

```
"""Token-aware chunking โดยใช้ tokenizer ตัวเดียวกับ embedding model
เพื่อรับประกันว่า chunk ไม่เกิน max sequence length ของโมเดลจริงๆ
(นับ token จริง ไม่ใช่ นับตัวอักษร - สำคัญมากสำหรับภาษาไทยที่ 1 คำ ≠ 1 token)
"""

import os
import re
from dataclasses import dataclass, field, asdict
from transformers import AutoTokenizer
import config

@dataclass
class Chunk:
    chunk_id: str
    doc_id: str
    text: str
    n_tokens: int
    metadata: dict = field(default_factory=dict)

class ThaiLegalChunker:
    def __init__(self, chunk_size=config.CHUNK_SIZE,
overlap=config.CHUNK_OVERLAP):
        self.tokenizer =
AutoTokenizer.from_pretrained(config.EMBEDDING_MODEL)
        self.chunk_size = chunk_size
        self.overlap = overlap
        # แยกตาม "มาตรา" ก่อน (structure-aware) แล้วค่อย pack เป็น chunk
        #self.section_pattern = re.compile(r"(?=มาตรา\s*[\do-๙]+)")
        self.section_pattern = re.compile(r"(?(?:มาตรา|ข้อ)\s*[\do-๙]+)")

    def _count_tokens(self, text: str) -> int:
        return len(self.tokenizer.encode(text, add_special_tokens=False))

    def _split_long_text(self, text: str) -> list[str]:
        """sliding window ระดับ token สำหรับข้อความที่ยาวเกิน chunk_size"""
        ids = self.tokenizer.encode(text, add_special_tokens=False)
```

```

        step = self.chunk_size - self.overlap
        pieces = []
        for start in range(0, len(ids), step):
            window = ids[start:start + self.chunk_size]
            pieces.append(self.tokenizer.decode(window,
skip_special_tokens=True))
            if start + self.chunk_size >= len(ids):
                break
        return pieces

    def chunk_document(self, doc_id: str, text: str) -> list[Chunk]:
        # 1) แยกตามมาตรา (ถ้าเจอ) - ไม่เจอก็ถือทั้งเอกสารเป็น section เดียว
        sections = [s.strip() for s in self.section_pattern.split(text) if
s.strip() ]
        chunks, buffer, buffer_tokens = [], [], 0

        def flush():
            nonlocal buffer, buffer_tokens
            if buffer:
                merged = "\n".join(buffer)
                chunks.append(merged)
                buffer, buffer_tokens = [], 0

        # 2) pack มาตราสั้นๆ รวมกันจนใกล้ chunk_size (ลดจำนวน chunk ที่สิ้นเกิน)
        for sec in sections:
            n = self._count_tokens(sec)
            if n > self.chunk_size:
                flush()
                chunks.extend(self._split_long_text(sec))
            elif buffer_tokens + n > self.chunk_size:
                flush()
                buffer, buffer_tokens = [sec], n
            else:
                buffer.append(sec)
                buffer_tokens += n
        flush()

        return [
            Chunk(
                chunk_id=f"{doc_id}::chunk_{i}",

```

```

        doc_id=doc_id,
        text=c,
        n_tokens=self._count_tokens(c),
    )
    for i, c in enumerate(chunks)
]

def chunk_corpus(self, docs_dir: str = config.DOCS_DIR) ->
list[Chunk]:
    all_chunks = []
    for fname in sorted(os.listdir(docs_dir)):
        if not fname.endswith(".txt"):
            continue
        with open(os.path.join(docs_dir, fname), encoding="utf-8") as
f:
            text = f.read()

    all_chunks.extend(self.chunk_document(fname.removesuffix(".txt"), text))
    return all_chunks

def chunks_to_dicts(chunks: list[Chunk]) -> list[dict]:
    return [asdict(c) for c in chunks]

```

-ไฟล์ [evaluation.py](#)

```

"""Evaluation Harness for Thai Legal RAG Pipeline
ประกอบด้วย:
1. BERTScore (xlm-roberta-large) สำหรับวัด Semantic Similarity
2. LLM-as-Judge แบบเข้มงวด (Claim-based Verification + Abstention Handling)
"""

from __future__ import annotations

import json
import re
from bert_score import score as bertscore

```

```

from openai import OpenAI
import config

# กำหนด Client เชื่อมต่อ Endpoint ของคลาส
judge_client = OpenAI(
    base_url=config.LLM_BASE_URL,
    api_key=config.LLM_API_KEY
)

# =====
# 1. BERTScore (Mandatory Metric 1)
# =====
def compute_bertscore(candidates: list[str], references: list[str]) -> dict:
    """คำนวณ BERTScore ด้วย xlm-roberta-large สำหรับภาษาไทย"""
    P, R, F1 = bertscore(
        candidates,
        references,
        model_type="xlm-roberta-large",
        lang="th",
        rescale_with_baseline=False,
    )
    return {
        "precision": float(P.mean()),
        "recall": float(R.mean()),
        "f1": float(F1.mean()),
    }

# =====
# 2. LLM-as-Judge (Mandatory Metric 2 - เข้มงวดพิเศษ)
# =====
JUDGE_PROMPT = """คุณเป็นกรรมการตรวจสอบระบบตอบคำถามกฎหมายไทย ที่เข้มงวดมาก

```

ขั้นตอนบังคับ:

1. แยกคำตอบออกเป็น "ข้อกล่าวอ้าง (claim)" ที่ละประโยค
2. ตรวจสอบทีละ claim ว่ามีข้อความรองรับในบริบทหรือไม่
3. รวบรวม claim ที่ "ไม่มีหลักฐานในบริบท" ไว้ใน unsupported_claims

เกณฑ์ Faithfulness (เข้มงวด - ห้ามให้ 5 ถ้าไม่เข้าใจจนไขเป๊ะ):

5 = ทุก claim มีหลักฐานตรงตัวในบริบท และเลขมาตรา/ข้อกฎหมายถูกต้องครบ
4 = claim หลักถูกทั้งหมด แต่มีการสรุปความเกินบริบทเล็กน้อย (ไม่ใช่ข้อเท็จจริงใหม่)
3 = มี 1 claim ที่ไม่มีหลักฐานรองรับ
2 = มีหลายจุดไม่มีหลักฐาน หรืออ้างเลขมาตราผิด/ไม่มีในบริบท
1 = เนื้อหาส่วนใหญ่แต่งขึ้นเอง

เกณฑ์ Relevance:

5 = ตอบตรงประเด็น ครบทุกส่วนของคำถาม
4 = ตอบตรงประเด็นครบถ้วน แต่มีส่วนขยายที่ไม่จำเป็น
3 = ตอบได้เพียงบางส่วนของคำถาม
2 = แตะประเด็นคำถามแต่ไม่ตอบสาระสำคัญ
1 = ไม่เกี่ยวข้องคำถาม

กรณีพิเศษ: ถ้าคำตอบคือการปฏิเสธ (เช่น "ไม่พบข้อมูลในเอกสารที่มี")

- ถ้าบริบท "ไม่มี" ข้อมูลคำตอบจริง -> faithfulness = 5, relevance = 5 (ปฏิเสธถูกต้อง)
- ถ้าบริบท "มี" ข้อมูลคำตอบอยู่แล้ว แต่ระบบหาไม่เจอ -> faithfulness = 5, relevance = 1 (ปฏิเสธผิดพลาด)

คำถาม: {query}

---บริบท---

{context}

---คำตอบของระบบ---

{answer}

ตอบเป็น JSON เท่านั้น ห้ามมีคำอธิบายอื่นนอกเหนือจาก JSON:

```
{{"faithfulness": <1-5>, "relevance": <1-5>, "unsupported_claims":  
["..."], "is_abstention": <true|false>, "reason": "<สั้นๆ>"}}
```

```
def _parse_json(txt: str) -> dict:  
    """ทำความสะอาดข้อความและตัด markdown backticks เพื่อแปลงเป็น JSON"""  
    t = str(txt).strip()  
    t = re.sub(r"^```(?:json)?|```$", "", t, flags=re.MULTILINE).strip()  
    try:  
        return json.loads(t)  
    except json.JSONDecodeError:  
        m = re.search(r"\{.*\}", t, re.DOTALL)  
        if m:  
            try:  
                return json.loads(m.group(0))
```

```

        except json.JSONDecodeError:
            pass
    return {}

def _clamp(v, lo=1, hi=5):
    """จำกัดคะแนนให้อยู่ในช่วง 1 - 5"""
    try:
        return max(lo, min(hi, int(round(float(v)))))
    except (TypeError, ValueError):
        return None

def llm_judge(query: str, context: str, answer: str) -> dict:
    """ประเมินคุณภาพคำตอบด้วย LLM-as-Judge พร้อมคืนค่าความสมบูรณ์และเหตุผล"""
    model_name = getattr(config, "JUDGE_MODEL", "gemma-4-E4B-it")
    temp = getattr(config, "JUDGE_TEMPERATURE", 0.0)

    try:
        resp = judge_client.chat.completions.create(
            model=model_name,
            temperature=temp,
            messages=[{
                "role": "user",
                "content": JUDGE_PROMPT.format(
                    query=query,
                    context=context[:6000], # ป้องกัน Context Window ล้น
                    answer=answer
                )
            }],
        )
        raw = _parse_json(resp.choices[0].message.content)
    except Exception as e:
        return {
            "faithfulness": None,
            "relevance": None,
            "unsupported_claims": [],
            "is_abstention": None,
            "reason": f"judge_error: {e}",
            "ok": False
        }

```

```

    }

    return {
        "faithfulness": _clamp(raw.get("faithfulness")),
        "relevance": _clamp(raw.get("relevance")),
        "unsupported_claims": raw.get("unsupported_claims") or [],
        "is_abstention": bool(raw.get("is_abstention", False)),
        "reason": str(raw.get("reason", ""))[:300],
        "ok": True,
    }

```

-ไฟล์ [extract_corpus.py](#)

```

import sys; sys.path.insert(0, ".")
import json
import os
from tqdm import tqdm
from src.extraction import extract_pdf
from pathlib import Path

# หา Root directory ของ thai-legal-rag โดยอิงจากไฟล์นี้
BASE_DIR = Path(__file__).resolve().parent.parent
PDF_DIR = "data/pdfs" # วาง PDF 80 ไฟล์ตรงนี้
OUT_DIR = "data/documents" # .txt ที่สะอาดแล้ว (chunker อ่านต่อจากตรงนี้)
os.makedirs(OUT_DIR, exist_ok=True)

reports = []
for fname in tqdm(sorted(os.listdir(PDF_DIR))):
    if not fname.lower().endswith(".pdf"):
        continue
    try:
        #text, report = extract_pdf(os.path.join(PDF_DIR, fname))
        # แปลงเป็น Absolute Path และใส่ prefix \\?\ ปลดล็อกข้อจำกัด 260 ตัวอักษรบน
Windows
        full_pdf_path = os.path.abspath(os.path.join(PDF_DIR, fname))
        if os.name == "nt" and not full_pdf_path.startswith("\\\\?\\"):
            full_pdf_path = "\\?\\\\" + full_pdf_path

```

```

        text, report = extract_pdf(full_pdf_path)
        out_name = fname[:-4] + ".txt"
        with open(os.path.join(OUT_DIR, out_name), "w", encoding="utf-8")
as f:
            f.write(text)
            reports.append(report)
        except Exception as e:
            reports.append({"file": fname, "method": "FAILED", "error":
str(e)})

with open("data/extraction_report.json", "w", encoding="utf-8") as f:
    json.dump(reports, f, ensure_ascii=False, indent=2)

# ---- สรุปให้เห็นภาพรวมทันที ----
ocr_files = [r["file"] for r in reports if r.get("method") == "ocr"]
failed = [r["file"] for r in reports if r.get("method") == "FAILED"]
suspicious = [r["file"] for r in reports
                if r.get("quality") and not r["quality"]["ok"]]
no_matra = [r["file"] for r in reports if r.get("n_matra", 1) == 0]

print(f"\n✅ สำเร็จ: {len(reports) - len(failed)}/{len(reports)}")
print(f"🔍 ใช้ OCR: {len(ocr_files)} ไฟล์ → {ocr_files[:5]}")
print(f"⚠️ คุณภาพน่าสงสัย (ต้องเปิดดูเอง!): {suspicious}")
print(f"⚠️ ไม่เจอคำว่า 'มาตรา' (schema ต่าง?): {no_matra}")
print(f"❌ พัง: {failed}")

```

-ไฟล์ [extraction.py](#)

```

"""PDF → clean text pipeline สำหรับเอกสารกฎหมายไทย

กลยุทธ์: ลอง text layer ก่อน (เร็ว) → ตรวจสอบคุณภาพ → ถ้าไม่ผ่านค่อย OCR (ช้าแต่ชัวร์)
เหตุผลที่ต้องมี quality check: PDF ไทยจำนวนมากมี text layer ที่ 'พัง'
(สระลอย/ตัวอักษรสลับ) ซึ่ง extract ได้โดยไม่ error แต่ embedding จะเสียทั้งหมด
"""

import io
import json
import os
import re

```

```

import unicodedata

#import fitz # PyMuPDF

import pymupdf as fitz
from PIL import Image
from pythainlp.util import normalize as thai_normalize

try:
    import pytesseract
    HAS_TESSERACT = True
    # ระบุ path ไปยังตัว tesseract.exe ในเครื่อง Windows
    if os.name == "nt": # เฉพาะบน Windows
        tess_path = r"C:\Program Files\Tesseract-OCR\tesseract.exe"
        if os.path.exists(tess_path):
            pytesseract.pytesseract.tesseract_cmd = tess_path
except ImportError:
    HAS_TESSERACT = False

# ----- Quality heuristics -----
THAI_CHAR = re.compile(r"[\u0E00-\u0E7F]")
# สระ/วรรณยุกต์ที่ "ลอย" (ตามหลังช่องว่าง) = สัญญาณ text layer พัง
FLOATING_MARKS = re.compile(r"\s[\u0E31\u0E34-\u0E3A\u0E47-\u0E4E]")

def quality_score(text: str) -> dict:
    """คืน metrics สำหรับตัดสินว่า text layer ใช้ได้ไหม"""
    if not text or len(text) < 50:
        return {"ok": False, "reason": "too_short", "thai_ratio": 0.0}

    no_space = re.sub(r"\s", "", text)
    thai_ratio = len(THAI_CHAR.findall(no_space)) / max(len(no_space), 1)
    floating = len(FLOATING_MARKS.findall(text))
    replacement = text.count("\ufffd") # อักขระที่ decode ไม่ได้

    ok = (
        thai_ratio > 0.5 # เอกสารไทยควรมีอักขระไทยเกินครึ่ง
        and floating < len(text) / 500 # สระลอยเกิน ~0.2% = layer พัง
        and replacement < 5
    )

```

```

reason = "ok" if ok else (
    "low_thai_ratio" if thai_ratio <= 0.5
    else "floating_marks" if floating >= len(text) / 500
    else "replacement_chars"
)
return {"ok": ok, "reason": reason, "thai_ratio": round(thai_ratio,
3),
        "floating_marks": floating}

# ----- Cleaning / Normalization -----
def clean_text(text: str) -> str:
    """Normalize ให้ทุกไฟล์มี 'schema กลาง' เดียวกันก่อนเข้า chunker"""
    # 1) Unicode normalization (สระ/วรรณยุกต์บางไฟล์ encode ต่างกัน)
    text = unicodedata.normalize("NFC", text)
    # 2) ลบ zero-width & soft hyphen ที่ PDF ขอบแทรก
    text = re.sub(r"[\u200b\u200c\u200d\u00ad\ufeff]", "", text)
    # 3) แปลงเลขหน้า/หัวกระดาษท้ายกระดาษที่พบบ่อยในราชกิจจานุเบกษา
    text = re.sub(r"^\s*หน้า\s*[\do-๙]+\s*$", "", text, flags=re.MULTILINE)
    text = re.sub(r"^\s*เล่ม\s*[\do-๙]+.*ราชกิจจานุเบกษา.*$", "", text,
flags=re.MULTILINE)
    text = re.sub(r"^\s*[--]?s*[\do-๙]+\s*[--]?s*$", "", text,
flags=re.MULTILINE)
    # 4) รวมบรรทัดที่ถูกตัดกลางประโยค (PDF ตัดบรรทัดตาม layout ไม่ใช่ตามความหมาย)
    #   แต่รักษา newline หน้า "มาตรา" ไว้ให้ chunker ใช้แยก section
    text = re.sub(r"\n(?:\s*(มาตรา|ข้อ|หมวด|ส่วนที่|บรรพ|ลักษณะ)\s)", " ",
text)
    # 5) ย่นช่องว่างซ้ำ
    text = re.sub(r"[ \t]{2,}", " ", text)
    text = re.sub(r"\n{3,}", "\n\n", text)
    # 6) ช่วยแก้สระซ้อน/สระลอย
    text = thai_normalize(text)
    return text.strip()

# ----- Extractors -----
def extract_text_layer(pdf_path: str) -> str:
    doc = fitz.open(pdf_path)
    pages = []
    for page in doc:

```

```

        # sort=True เรียง block ตามตำแหน่งบนหน้า (แก้ปัญหาลำดับข้อความสลับ)
        pages.append(page.get_text("text", sort=True))
    doc.close()
    return "\n".join(pages)

def extract_ocr(pdf_path: str, dpi: int = 300) -> str:
    """Render แต่ละหน้าเป็นภาพแล้ว OCR - ใช้เมื่อ text layer ใช้ไม่ได้"""
    assert HAS_TESSERACT, "ต้องติดตั้ง pytesseract + tesseract-ocr-tha ก่อน"
    doc = fitz.open(pdf_path)
    pages = []
    zoom = dpi / 72
    for page in doc:
        pix = page.get_pixmap(matrix=fitz.Matrix(zoom, zoom))
        img = Image.open(io.BytesIO(pix.tobytes("png")))
        # tha+eng เพราะเอกสารกฎหมายมักมีเลขอารบิก/ชื่อเฉพาะภาษาอังกฤษปน
        pages.append(pytesseract.image_to_string(img, lang="tha+eng"))
    doc.close()
    return "\n".join(pages)

def extract_pdf(pdf_path: str) -> tuple[str, dict]:
    """Entry point: คืน (cleaned_text, report)"""
    raw = extract_text_layer(pdf_path)
    q = quality_score(raw)
    method = "text_layer"

    if not q["ok"] and HAS_TESSERACT:
        raw_ocr = extract_ocr(pdf_path)
        q_ocr = quality_score(raw_ocr)
        #if q_ocr["thai_ratio"] > q["thai_ratio"]: # ใช้อันที่ดีกว่า
        if q_ocr["ok"] or len(raw.strip()) < 50: #ถ้า OCR ผ่านเกณฑ์ หรือเดิม
text layer มีข้อความน้อยมากๆ ให้เลือก OCR
            raw, q, method = raw_ocr, q_ocr, "ocr"

    cleaned = clean_text(raw)
    report = {
        "file": os.path.basename(pdf_path),
        "method": method,
        "quality": q,

```

```

        "n_chars": len(cleaned),
        "n_matra": len(re.findall(r"มาตรา\s*[\do-๙]+", cleaned)),
    }
    return cleaned, report

```

-ไฟล์ [generation.py](#)

```

"""Generation Module with Abstention Handling for Thai Legal Q&A
ระบบสร้างคำตอบทางกฎหมาย พร้อมกลไกปฏิเสธการตอบ (Abstain) เพื่อป้องกัน Hallucination
"""
from __future__ import annotations

from openai import OpenAI
import config

# ใช้ Endpoint และ Key ของคลาสดตาม config
client = OpenAI(
    base_url=config.LLM_BASE_URL,
    api_key=config.LLM_API_KEY
)

# ดึงข้อความปฏิเสธมาตรฐานจาก config
ABSTAIN_TEXT = getattr(config, "ABSTAIN_TEXT", "ไม่พบข้อมูลในเอกสารที่มี")

SYSTEM_PROMPT = f"""คุณเป็นผู้ช่วยตอบคำถามกฎหมายไทย ตอบโดยอ้างอิง "บริบทที่ให้มาเท่านั้น"

กฎเหล็ก:
1. ห้ามใช้ความรู้นอกบริบท ห้ามเดา ห้ามแต่งเลขมาตราหรือชื่อพระราชบัญญัติ
2. ถ้าบริบทไม่มีข้อมูลเพียงพอ ให้ตอบเพียงว่า "{ABSTAIN_TEXT}" แล้วหยุดทันที
3. ถ้าตอบได้บางส่วน ให้ตอบเฉพาะส่วนที่มีหลักฐาน แล้วระบุชัดเจนว่าส่วนใดไม่พบข้อมูล
4. อ้างอิงเลขมาตราและชื่อเอกสารที่ปรากฏในบริบทเสมอ
5. ตอบเป็นภาษาไทย กระชับ ตรงประเด็น
"""

ABSTAIN_MARKERS = (
    "ไม่พบข้อมูล",
    "ไม่มีข้อมูล",

```

```

        "ไม่สามารถตอบ",
        "ไม่ปรากฏใน",
        "บริบทไม่ได้รับ",
        "ไม่ปรากฏข้อมูล"
    )

def is_abstention(answer: str) -> bool:
    """ตรวจสอบว่าคำตอบเป็นการปฏิเสธ (Abstain) หรือไม่ ตรวจสอบเฉพาะช่วง 120 ตัวอักษรแรก"""
    a = str(answer or "").strip()
    if not a:
        return True
    head = a[:120]
    return any(m in head for m in ABSTAIN_MARKERS)

def build_context(contexts: list[dict]) -> str:
    """จัดรูปแบบ Context Block ให้มีป้ายกำกับชัดเจน เพื่อช่วยให้ LLM สังเคราะห์คำตอบได้แม่นยำ"""
    parts = []
    for i, c in enumerate(contexts, 1):
        doc = c.get("doc_id") or c.get("file") or "เอกสารอ้างอิง"
        text = c.get("text", "").strip()
        parts.append(f"บริบท {i} | แหล่งที่มา: {doc}]\n{text}")
    return "\n\n---\n\n".join(parts)

def generate_answer(query: str, contexts: list[dict]) -> str:
    """สร้างคำตอบจากบริบทที่ได้รับ หากไม่มีบริบทจะ Abstain ทันทีโดยไม่มี API"""
    if not contexts:
        return ABSTAIN_TEXT

    model_name = getattr(config, "LLM_MODEL", getattr(config, "GEN_MODEL",
"gemma-4-E4B-it"))

    resp = client.chat.completions.create(
        model=model_name,
        temperature=0.0, # ใช้ 0.0 เพื่อความแม่นยำสูงสุด ลดการสุ่มแต่งคำตอบ
        messages=[

```

```

        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": f"บริบท:\n{build_context(contexts)}\n\nคำถาม: {query}"},
    ],
)
return (resp.choices[0].message.content or "").strip()

def generate_answer_stream(query: str, contexts: list[dict]):
    """สร้างคำตอบแบบ Streaming คืนค่าทีละ Token (Yield Chunks)"""
    if not contexts:
        yield ABSTAIN_TEXT
        return

    model_name = getattr(config, "LLM_MODEL", getattr(config, "GEN_MODEL",
"gemma-4-E4B-it"))

    response = client.chat.completions.create(
        model=model_name,
        temperature=0.0,
        stream=True, # ★ เปิด Streaming จาก OpenAI Endpoint
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": f"บริบท:\n{build_context(contexts)}\n\nคำถาม: {query}"},
        ],
    )
    for chunk in response:
        content = chunk.choices[0].delta.content
        if content:
            yield content

```

-ไฟล์ [indexer.py](#)

```

import json
import os
import pickle
import numpy as np
import faiss
from sentence_transformers import SentenceTransformer
from rank_bm25 import BM25Okapi

```

```

from pythainlp.tokenize import word_tokenize
import config

def thai_tokenize(text: str) -> list[str]:
    """ตัดคำภาษาไทยด้วย newmm - จำเป็นเพราะ BM25 ต้องการ word-level tokens
    (ภาษาไทยไม่มีช่องว่างคั่นคำ ใช้ split() ตรงๆ ไม่ได้)"""
    return [t for t in word_tokenize(text, engine="newmm") if t.strip()]

class Indexer:
    def __init__(self):
        self.model = SentenceTransformer(config.EMBEDDING_MODEL,
device="cpu")
        # กำหนดความยาวสูงสุดของโมเดลตรงนี้ (แก้ปัญหา Sequence เกิน 512 tokens)
        self.model.max_seq_length = 512

    def build(self, chunks: list[dict], out_dir: str = config.INDEX_DIR):
        os.makedirs(out_dir, exist_ok=True)
        texts = [c["text"] for c in chunks]

        # ---- Dense (E5 ต้องใส่ prefix "passage: " ตอน index) ----
        embeddings = self.model.encode(
            [f"passage: {t}" for t in texts],
            batch_size=16,
            normalize_embeddings=True, # normalize → inner product =
cosine similarity
            show_progress_bar=True,
            #max_length=512, # กำหนดไม่ให้เกิน Sequence length สูงสุดของ
e5-base
            #truncation=True, # ตัด token ส่วนเกินทิ้งอัตโนมัติหากเกิน 512
        ).astype("float32")

        index = faiss.IndexFlatIP(embeddings.shape[1])
        index.add(embeddings)
        faiss.write_index(index, os.path.join(out_dir, "dense.faiss"))

        # ---- BM25 (สำหรับ hybrid variant) ----
        tokenized = [thai_tokenize(t) for t in texts]
        bm25 = BM25Okapi(tokenized)

```

```

        with open(os.path.join(out_dir, "bm25.pkl"), "wb") as f:
            pickle.dump(bm25, f)

        with open(os.path.join(out_dir, "chunks.json"), "w",
encoding="utf-8") as f:
            json.dump(chunks, f, ensure_ascii=False, indent=1)

        print(f"Indexed {len(chunks)} chunks → {out_dir}")

```

-ไฟล์ [metrics.py](#)

```

"""Information Retrieval Metrics & Statistical Significance Harness
โมดูลสำหรับคำนวณ Retrieval Metrics (Recall@k, MRR@k, nDCG@k, Hit@k)
พร้อมระบบแก้ปัญหา Document ID String Mismatch และ Bootstrap Statistical Tests
"""

from __future__ import annotations

import math
import random
from typing import Sequence

def norm_doc(x) -> str:
    """Normalize ชื่อเอกสารเพื่อแก้ปัญหา String Mismatch
    ตัด path ทั้งแบบ Windows (\) และ POSIX (/), ตัดนามสกุลไฟล์ (.txt, .pdf ฯลฯ),
และตัด whitespace
    """
    if x is None:
        return ""
    s = str(x).strip().replace("\\", "/").split("/")[-1].lower()
    for ext in (".txt", ".pdf", ".json", ".docx"):
        if s.endswith(ext):
            s = s[: -len(ext)]
    return s.strip()

def gold_docs_of(item: dict) -> set[str]:
    """ดึงรายชื่อ Ground Truth Document ID จาก testset.json
    รองรับทุกรูปแบบ Key ทั้ง relevant_docs (list), source_doc (str), doc_id ฯลฯ
    """

```

```

"""
for key in ("relevant_docs", "gold_docs", "relevant_doc_ids"):
    v = item.get(key)
    if v:
        v = [v] if isinstance(v, str) else v
        out = {norm_doc(d) for d in v if norm_doc(d)}
        if out:
            return out
for key in ("source_doc", "doc_id", "source"):
    v = item.get(key)
    if v and norm_doc(v):
        return {norm_doc(v)}
return set()

def retrieved_docs_of(hits: Sequence[dict]) -> list[str]:
    """ดึงรายชื่อ Document ID จาก Chunks ที่ Retriever ดึงขึ้นมา พร้อม Normalize"""
    docs = []
    for h in hits:
        # รวบรวมทั้ง doc_id, file, source หรือ parse จาก chunk_id (เช่น
"doc_name::chunk_0")
        raw_id = h.get("doc_id") or h.get("file") or h.get("source")
        if not raw_id and "chunk_id" in h:
            raw_id = str(h["chunk_id"]).rsplit("::", 1)[0]
        docs.append(norm_doc(raw_id))
    return docs

# =====
# 1. Retrieval Metrics
# =====

def recall_at_k(ret: Sequence[str], gold: set[str], k: int = 5) -> float:
    """สัดส่วนของเอกสารที่ถูกต้อง (Gold Docs) ที่ถูกค้นพบใน Top-K"""
    if not gold:
        return 0.0
    pool = ret[:k]
    found = set(pool) & gold
    return len(found) / len(gold)

```

```

def hit_at_k(ret: Sequence[str], gold: set[str], k: int = 5) -> float:
    """ตรวจสอบว่าเจอเอกสารที่ถูกต้องอย่างน้อย 1 รายการใน Top-K หรือไม่ (Hit Rate)"""
    if not gold:
        return 0.0
    return 1.0 if set(ret[:k]) & gold else 0.0

def mrr(ret: Sequence[str], gold: set[str], k: int | None = 5) -> float:
    """Mean Reciprocal Rank (MRR): คำนวณส่วนกลับของลำดับแรกที่พบเอกสารถูกต้อง (1/Rank)"""
    if not gold:
        return 0.0
    pool = ret[:k] if k is not None else ret
    for rank, d in enumerate(pool, start=1):
        if d in gold:
            return 1.0 / rank
    return 0.0

def ndcg_at_k(ret: Sequence[str], gold: set[str], k: int = 5) -> float:
    """Normalized Discounted Cumulative Gain (nDCG@K) สำหรับวัด Ranking Quality"""
    if not gold:
        return 0.0
    pool = ret[:k]
    dcg = sum(1.0 / math.log2(i + 1) for i, d in enumerate(pool, start=1)
    if d in gold)
    # Ideal DCG: สมมติว่าเอกสารที่ถูกต้องทั้งหมดขึ้นมายู่อันดับบนสุด
    ideal_hits = min(len(gold), k)
    ideal = sum(1.0 / math.log2(i + 1) for i in range(1, ideal_hits + 1))
    return dcg / ideal if ideal > 0.0 else 0.0

# =====
# 2. สถิติเชิงอนุมาน (Statistical Tests)
# =====

def mean(v: Sequence[float]) -> float:
    """คำนวณค่าเฉลี่ย โดยกรองค่า NaN ออกอัตโนมัติ"""

```

```

vals = [x for x in v if x == x and x is not None]
return sum(vals) / len(vals) if vals else 0.0

def bootstrap_ci(values: Sequence[float], n_boot: int = 2000, alpha: float
= 0.05, seed: int = 42) -> tuple[float, float]:
    """คำนวณ 95% Bootstrap Confidence Interval ของค่าเฉลี่ย"""
    vals = [x for x in values if x == x and x is not None]
    n = len(vals)
    if n == 0:
        return (0.0, 0.0)
    rng = random.Random(seed)
    boot_means = sorted(
        sum(vals[rng.randrange(n)] for _ in range(n)) / n
        for _ in range(n_boot)
    )
    lo_idx = max(0, int((alpha / 2) * n_boot))
    hi_idx = min(n_boot - 1, int((1 - alpha / 2) * n_boot))
    return (boot_means[lo_idx], boot_means[hi_idx])

def paired_bootstrap_p(a: Sequence[float], b: Sequence[float], n_boot: int
= 2000, seed: int = 42) -> float:
    """Paired Bootstrap Hypothesis Test
    H0: mean(a) == mean(b) - ทดสอบว่า Variant C (a) เหนือกว่า Variant A (b)
อย่างมีนัยสำคัญหรือไม่
    """
    pairs = [(x, y) for x, y in zip(a, b) if x == x and y == y and x is
not None and y is not None]
    if not pairs:
        return 1.0

    diffs = [x - y for x, y in pairs]
    n = len(diffs)
    obs = sum(diffs) / n
    if obs == 0:
        return 1.0

    centered = [d - obs for d in diffs]
    rng = random.Random(seed)

```

```

    cnt = sum(
        1 for _ in range(n_boot)
        if abs(sum(centered[rng.randrange(n)] for _ in range(n)) / n) >=
abs(obs)
    )
    return (cnt + 1) / (n_boot + 1)

```

-ไฟล์ [pipeline.py](#)

```

from src.retrieval import Retriever
from src.generation import generate_answer, generate_answer_stream

class RAGPipeline:
    def __init__(self, mode: str = "hybrid_rerank"):
        self.mode = mode
        self.retriever = Retriever(use_reranker=(mode == "hybrid_rerank"))

    def answer(self, query: str) -> dict:
        contexts = self.retriever.retrieve(query, mode=self.mode)
        answer = generate_answer(query, contexts)
        return {
            "query": query,
            "answer": answer,
            "contexts": contexts,
            "mode": self.mode,
        }

    # def answer_stream(self, query: str):
    #     """ค้นหาบริบทก่อน แล้วสตรีมคำตอบออกมาทีละ token"""
    #     contexts = self.retriever.retrieve(query, mode=self.mode)
    #     return contexts, generate_answer_stream(query, contexts)
    def answer_stream(self, query: str, retrieve_k: int | None = None):
        """ค้นหาบริบทก่อน แล้วสตรีมคำตอบออกมาทีละ token"""
        # ส่ง retrieve_k ต่อไปยังตัว retriever
        contexts = self.retriever.retrieve(query, mode=self.mode,
retrieve_k=retrieve_k)
        return contexts, generate_answer_stream(query, contexts)

```

-ไฟล์ [retrieval.py](#)

```
"""Retrieval Pipeline: Dense, BM25, Hybrid (RRF), and Cross-Encoder
Reranking
รองรับการดึงข้อมูลแบบ 2-Stage Retrieval พร้อมระบบ Candidate Pool สำหรับ Reranker
"""
from __future__ import annotations

import json
import pickle
import threading
from pathlib import Path
from typing import Any

import faiss
import numpy as np
from rank_bm25 import BM25Okapi
from sentence_transformers import SentenceTransformer, CrossEncoder

from src.indexer import thai_tokenize
import config

VALID_MODES = ("dense", "bm25", "hybrid", "hybrid_rerank")

class Retriever:
    _reranker = None
    _lock = threading.Lock()

    def __init__(self, index_dir: str | Path | None = None, use_reranker:
bool = False):
        # รองรับทั้ง index_dir เดิม และ artifacts_dir
        target_dir = index_dir or getattr(config, "ARTIFACTS_DIR",
getattr(config, "INDEX_DIR", "artifacts"))
        art = Path(target_dir)

        # 1. โหลด Chunks
        with open(art / "chunks.json", encoding="utf-8") as f:
            self.chunks: list[dict] = json.load(f)
```

```

# 2. โหลด FAISS Dense Index
self.index = faiss.read_index(str(art / "dense.faiss"))

# 3. โหลด BM25 Model (รองรับทั้ง object ใดๆ และ dictionary wrapper)
with open(art / "bm25.pkl", "rb") as f:
    obj = pickle.load(f)
    if isinstance(obj, dict):
        self.bm25 = obj.get("bm25") or obj.get("model")
        if self.bm25 is None:
            corpus = obj.get("tokenized") or obj.get("corpus")
            if corpus is None:
                raise ValueError("bm25.pkl ไม่มีทั้ง model และ corpus")
            self.bm25 = BM25Okapi(corpus)
    else:
        self.bm25 = obj

# Data Guard: ตรวจสอบความสอดคล้องของ FAISS Index กับ chunks.json
if self.index.ntotal != len(self.chunks):
    raise RuntimeError(
        f"index ntotal={self.index.ntotal} !=
chunks={len(self.chunks)} "
        "-> artifacts ไม่ sync กัน ให้รัน build_index.py ใหม่"
    )

# 4. โหลด Dense Embedding Model
self.model = SentenceTransformer(config.EMBEDDING_MODEL,
device="cpu")

# ----- Helper Methods -----
def _hit(self, idx: int, score: float, rank: int) -> dict:
    c = dict(self.chunks[idx])
    c["score"] = float(score)
    c["rank"] = rank
    c["_idx"] = int(idx)
    c.setdefault("doc_id", str(c.get("chunk_id", "")).rsplit("::",
1)[0])
    return c

@classmethod
def _get_reranker(cls) -> CrossEncoder:

```

```

        with cls._lock:
            if cls._reranker is None:
                cls._reranker = CrossEncoder(config.RERANKER_MODEL,
max_length=512, device="cpu")
            return cls._reranker

# ----- Search Methods -----
def _dense_search(self, query: str, k: int) -> list[dict]:
    k = max(0, min(int(k), self.index.ntotal))
    if k == 0:
        return []
    prefix = getattr(config, "E5_QUERY_PREFIX", "query: ")
    q = self.model.encode(
        [prefix + query],
        normalize_embeddings=True,
        convert_to_numpy=True,
    ).astype("float32")
    scores, ids = self.index.search(q, k)
    return [
        self._hit(i, s, r)
        for r, (i, s) in enumerate(zip(ids[0], scores[0]), 1)
        if i >= 0
    ]

def _bm25_search(self, query: str, k: int) -> list[dict]:
    toks = thai_tokenize(query)
    if not toks:
        return []
    scores = np.asarray(self.bm25.get_scores(toks), dtype="float64")
    n_docs = scores.shape[0]
    k = max(0, min(int(k), n_docs))
    if k == 0:
        return []

    # ป้องกัน Index Out of Bounds และจัดเรียงอันดับคะแนนคงที่
    if k < n_docs:
        top_idx = np.argpartition(-scores, k - 1)[:k]
        top = top_idx[np.argsort(-scores[top_idx], kind="stable")]
    else:
        top = np.argsort(-scores, kind="stable")

```

```

        return [self._hit(int(i), float(scores[i]), r) for r, i in
enumerate(top, 1)]

    @staticmethod
    def _rrf(rank_lists: list[list[dict]], k_const: int = 60) ->
list[dict]:
        fused: dict[Any, dict] = {}
        for lst in rank_lists:
            for rank, hit in enumerate(lst, 1):
                cid = hit["chunk_id"]
                e = fused.setdefault(cid, {"hit": hit, "score": 0.0})
                e["score"] += 1.0 / (k_const + rank)
        out = []
        for cid, e in fused.items():
            h = dict(e["hit"])
            h["score"] = e["score"]
            out.append(h)
        out.sort(key=lambda x: (-x["score"], str(x["chunk_id"])))
        for r, h in enumerate(out, 1):
            h["rank"] = r
        return out

    def rerank(self, query: str, candidates: list[dict], top_n: int) ->
list[dict]:
        if not candidates:
            return []
        ce = self._get_reranker()
        pairs = [(query, str(c.get("text", ""))) for c in candidates]
        batch_size = getattr(config, "RERANK_BATCH_SIZE", 16)
        scores = ce.predict(pairs, batch_size=batch_size,
show_progress_bar=False)
        out = []
        for c, s in zip(candidates, scores):
            d = dict(c)
            d["pre_rerank_rank"] = c.get("rank")
            d["rerank_score"] = float(s)
            out.append(d)
        out.sort(key=lambda x: (-x["rerank_score"], str(x["chunk_id"])))
        for r, d in enumerate(out, 1):

```

```

        d["rank"] = r
        return out[:top_n]

# ----- Public API -----
def retrieve(
    self,
    query: str,
    mode: str = "hybrid",
    k: int | None = None,
    retrieve_k: int | None = None,
    return_debug: bool = False,
) -> list[dict] | tuple[list[dict], dict]:
    if mode not in VALID_MODES:
        raise ValueError(f"mode ต้องเป็นหนึ่งใน {VALID_MODES}, ได้ {mode!r}")
    if not str(query).strip():
        return ([], {"error": "empty query"}) if return_debug else []

    final_k = int(k if k is not None else getattr(config,
"TOP_K_FINAL", 5))

    # จัดการขนาด Candidate Pool ก่อน Rerank หรือ Fusion
    if retrieve_k is None:
        default_pool = getattr(config, "TOP_K_RETRIEVE", 30)
        retrieve_k = getattr(config, "RERANK_CANDIDATES",
default_pool) if mode == "hybrid_rerank" else default_pool

    pool = max(int(retrieve_k), final_k)
    rrf_k = getattr(config, "RRF_K", 60)
    dbg: dict[str, Any] = {"mode": mode, "pool": pool, "final_k":
final_k}

    if mode == "dense":
        results = self._dense_search(query, pool)[:final_k]
    elif mode == "bm25":
        results = self._bm25_search(query, pool)[:final_k]
    elif mode == "hybrid":
        d = self._dense_search(query, pool)
        b = self._bm25_search(query, pool)
        dbg.update(n_dense=len(d), n_bm25=len(b))

```

```

        results = self._rrf([d, b], rrf_k)[:final_k]
    else: # hybrid_rerank
        d = self._dense_search(query, pool)
        b = self._bm25_search(query, pool)
        cand = self._rrf([d, b], rrf_k)[:pool]
        dbg.update(n_dense=len(d), n_bm25=len(b),
n_candidates=len(cand))
        results = self.rerank(query, cand, final_k)

    dbg["n_returned"] = len(results)
    return (results, dbg) if return_debug else results

```

-ไฟล์ [smoke_test.py](#)

```

"""Smoke Test & System Sanity Check
สคริปต์ตรวจสอบความพร้อมของระบบแบบครบวงจร (Configuration, Metrics, Testset, และ
Live Retriever)
ก่อนเริ่มรันสคริปต์ประเมินผลจริง (scripts/run_ablation.py และ scripts/run_eval.py)
"""
from __future__ import annotations

import json
import sys
from pathlib import Path

ROOT_DIR = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT_DIR))

import config
from src.metrics import norm_doc, gold_docs_of, recall_at_k, hit_at_k,
mrr, ndcg_at_k

PASS, FAIL = "✅", "❌"
errors = []

def check(name: str, cond: bool):

```

```

print(f"{PASS if cond else FAIL} {name}")
if not cond:
    errors.append(name)

print("— 1. Configuration Sanity —")
top_k_final = getattr(config, "TOP_K_FINAL", 5)
top_k_retrieve = getattr(config, "TOP_K_RETRIEVE", 30)
rerank_cands = getattr(config, "RERANK_CANDIDATES", top_k_retrieve)
eval_k_vals = getattr(config, "EVAL_K_VALUES", [1, 3, 5])
max_eval_k = getattr(config, "MAX_EVAL_K", max(eval_k_vals))

check("Candidate Pool > TOP_K_FINAL", rerank_cands > top_k_final)
check("TOP_K_RETRIEVE >= MAX_EVAL_K", top_k_retrieve >= max_eval_k)
check("Endpoint URL Configured", bool(getattr(config, "LLM_BASE_URL",
"")))
check("Model Configured", bool(getattr(config, "LLM_MODEL", "")))

print("\n— 2. Information Retrieval Metrics —")
check("norm_doc ดัชนีเอกสารและพาร", norm_doc("data/docs/A.TXT") == "a")
check("gold จาก source_doc", gold_docs_of({"source_doc": "civil.txt"}) ==
{"civil"})
check("gold จาก relevant_docs", gold_docs_of({"relevant_docs": ["a.txt",
"b.txt"]}) == {"a", "b"})

sample_ret, sample_gold = ["x", "a", "y", "b", "z"], {"a", "b"}
check("recall@5 = 1.0", recall_at_k(sample_ret, sample_gold, 5) == 1.0)
check("recall@1 = 0.0", recall_at_k(sample_ret, sample_gold, 1) == 0.0)
check("hit@2 = 1.0", hit_at_k(sample_ret, sample_gold, 2) == 1.0)
check("mrr = 0.5", abs(mrr(sample_ret, sample_gold, 5) - 0.5) < 1e-9)
check("ndcg@5 อยู่ใน [0, 1]", 0 <= ndcg_at_k(sample_ret, sample_gold, 5) <=
1)
check("gold วาง คำนวณ 0.0 ปลอดภัย", recall_at_k(sample_ret, set(), 5) ==
0.0)

print("\n— 3. Testset & Corpus Sync —")
testset_path = getattr(config, "TESTSET_PATH", ROOT_DIR / "data" /
"testset.json")
artifacts_dir = Path(getattr(config, "ARTIFACTS_DIR", getattr(config,
"INDEX_DIR", ROOT_DIR / "artifacts")))

```

```

with open(testset_path, encoding="utf-8") as f:
    ts = json.load(f)
ts = ts.get("questions", ts) if isinstance(ts, dict) else ts

adv = [t for t in ts if t.get("adversarial")]
normal = [t for t in ts if not t.get("adversarial")]

check(f"มี Adversarial Questions >= 5 ข้อ (พบ {len(adv)} ข้อ)", len(adv) >=
5)
check("คำถามปกติทุกข้อมี Gold Docs ครอบคลุม", all(gold_docs_of(t) for t in
normal))

with open(artifacts_dir / "chunks.json", encoding="utf-8") as f:
    chunk_data = json.load(f)
chunk_docs = {norm_doc(c.get("doc_id") or c.get("file") or
str(c.get("chunk_id", "")).rsplit("::", 1)[0]) for c in chunk_data}

orphan = [t.get("id") for t in normal if not (gold_docs_of(t) &
chunk_docs)]
check(f"เอกสารอ้างอิงทุกข้อมีอยู่ใน Corpus (Orphans: {orphan[:3]})", not orphan)

print("\n— 4. Live Retriever Tests —")
from src.retrieval import Retriever

r = Retriever(index_dir=artifacts_dir)
sample_query = normal[0].get("question") or normal[0].get("query")

for m in ("dense", "bm25", "hybrid"):
    hits = r.retrieve(sample_query, mode=m, k=5)
    check(f"Retriever mode='{m}' คืนค่า 5 Chunks", len(hits) == 5)

hits, dbg = r.retrieve(
    sample_query,
    mode="hybrid_rerank",
    k=5,
    retrieve_k=rerank_cands,
    return_debug=True,
)

```

```

check(f"Candidate ป้อนเข้า Reranker > 5 (ได้ {dbg.get('n_candidates',
len(hits))})", dbg.get("n_candidates", 0) > 5)
check("ผลลัพธ์สุดท้าย hybrid_rerank = 5", len(hits) == 5)
check("ทุก Chunk มี rerank_score", all("rerank_score" in h for h in hits))
check(
    "คะแนน rerank_score เรียงจากมากไปน้อย",
    all(hits[i]["rerank_score"] >= hits[i + 1]["rerank_score"] for i in
range(len(hits) - 1)),
)

check("Query วางเปล่าไม่ Crash", r.retrieve(" ", mode="hybrid") == [])

try:
    r.retrieve(sample_query, mode="invalid_mode")
    check("Mode ผิดต้อง Raise ValueError", False)
except ValueError:
    check("Mode ผิดต้อง Raise ValueError", True)

print("\n" + ("=" * 50))
if not errors:
    print(f"{PASS} ผ่านการทดสอบทั้งหมด ระบบพร้อมรัน run_ablation.py และ
run_eval.py")
else:
    print(f"{FAIL} พบปัญหา {len(errors)} รายการที่ต้องแก้ไขก่อนรันจริง: {errors}")

sys.exit(1 if errors else 0)

```

ไฟล์ .gitignore

```

# Python & Cache
__pycache__/
*.py[cod]
*$py.class
.pytest_cache/

```

```
# Virtual Environment
.venv/
env/
venv/
ENV/

# Sensitive Configs / Credentials
# (หากยังฮาร์ดโค้ด Key ใน config.py ให้เปิดคอมเมนต์บรรทัดล่างนี้)
# config.py
.env
*.pem
*.key

# Binary Index Artifacts (ขนาดใหญ่ และสร้างใหม่ได้จาก scripts/build_index.py)
artifacts/*.faiss
artifacts/*.pkl

# OS specific files
.DS_Store
Thumbs.db
desktop.ini
```

ไฟล์ AI_AUDIT.md

AI Audit

****Project:**** Domain-Specific Intelligent Q&A System – Thai Legal RAG Pipeline

****Team:****

#	Name	Student ID
1	ณภัทร วาณิชรัตถากร	67070226
2	พลาริป เหมวุฒิ	67070255
3	อองรักษ์ วณิชขานัย	67070297
4	อ้ออังกูร ชัยวิวัฒน์พร	67070302

****AI Tools Used:**** Claude Fable 5, Gemini, Claude Sonnet 5

```

---
## 1. Tool Inventory

| AI Tool | Primary Use in This Project |
|---|---|
| Claude Fable 5 | ออกแบบสถาปัตยกรรมระบบ RAG และสร้างโค้ด Baseline Full Pipeline (Ingestion, Chunking, Hybrid Retrieval, Evaluation, Ablation) |
| Gemini | ตรวจสอบและแก้ไขบั๊กเชิงลึกระดับ Runtime (Windows MAX_PATH limit, Ingestion crash, Evaluation schema mismatch, Cross-platform scripts) |
| Claude Sonnet 5 | จัดทำโครงสร้างเอกสารทางเทคนิค บันทึกผลการทดลอง และจัดรูปแบบ Markdown สำหรับ README.md และ AI_AUDIT.md |

---

## 2. Prompt Log

### Entry 1

**Prompt sent:**
> เนื้อคือโปรเจกต์ เกี่ยวกับ RAG ผมอยากให้คุณช่วยเขียนโค้ดสร้าง RAG แบบ Full Pipeline จบเลยได้ไหม ? โดยมีรายละเอียดตามในไฟล์ PDF นี้เลย ส่วน Domain ที่ผมเลือกคือ กฎหมายและตอนนี้ผมเตรียม dataset พร้อมแล้วมีประมาณ 80 Document และเป็นภาษาไทยทั้งหมด

**What AI generated:**
สถาปัตยกรรมระบบ Thai Legal RAG แบบครบวงจร ประกอบด้วย Token-aware chunking, Dense retrieval ด้วยโมเดล SentenceTransformers (`multilingual-e5-base`), BM25 + RRF Hybrid merging, Cross-encoder Reranker (`bge-reranker`), และสคริปต์ประเมินผล BERTScore + LLM-as-Judge

**What you changed or rejected:**
แก้ไขปัญหา Path ในสคริปต์ `src/extract_corpus.py` โดยเพิ่มการคำนวณ Absolute Path อิงจาก Root Directory (`BASE_DIR = Path(__file__).resolve().parent.parent`) และแยกโครงสร้างโฟลเดอร์ให้ชัดเจนระหว่างไฟล์ต้นทางและปลายทาง (`PDF_DIR = "data/pdfs"`, `OUT_DIR = "data/documents"`) เพื่อไม่ให้ Chunker สับสนระหว่างไฟล์ `.pdf` กับ `.txt`

---

### Entry 2

```

****Prompt sent:****

```

```
py -c "import json;
r=json.load(open('data/extraction_report.json',encoding='utf-8'));
[print(x['file'][:50],'->',x.get('error')) for x in r if
x.get('method')=='FAILED']"
```

ประกาศกระทรวงดิจิทัล... -> no such file: 'data/pdfs\ประกาศกระทรวงดิจิทัล...pdf'

ประกาศคณะกรรมการการรักษาความมั่นคงปลอดภัยไซเบอร์... -> no such file...

ทำไมถึงเปิดไฟล์ไม่ได้ทั้งที่ไฟล์มีอยู่จริง?

```

****What AI generated:****

วินิจฉัยว่าติดข้อจำกัดความยาว Path บนระบบปฏิบัติการ Windows (MAX_PATH = 260 characters) เนื่องจากชื่อไฟล์ประกาศกฎหมายมีความยาวเกิน 150 ตัวอักษร และแนะนำให้ใส่ Prefix `\\?\` หน้า Absolute Path ในโค้ด Python

****What you changed or rejected:****

ปฏิเสธแนวทางการใช้ `\\?\` prefix ในโค้ด เพราะพบว่าไลบรารี C/C++ ภายในของ PyMuPDF (`fitz.open()`) ไม่รองรับ Windows Extended Path Prefix ทำให้เกิด Crash และ FileNotFoundError ต่อเนื่อง จึงเปลี่ยนไปแก้ปัญหที่ระดับโครงสร้างไฟล์โดยตรง คือ ย้ายโปรเจกต์ไปยัง Root Directory ที่สั้นลง (`D:\thai-legal-rag`) และ Rename ชื่อไฟล์ PDF ภาษาไทยที่ยาวเกินขนาดให้กระชับ เพื่อคงความเข้ากันได้แบบ Cross-platform

Entry 3

****Prompt sent:****

> ช่วยสร้าง testset.json โดยอ้างอิงจากข้อมูลในเอกสารกฎหมายเหล่านี้หน่อย ผมอยากเห็นสัก 20 testset [แนบไฟล์ PDF กฎหมาย 6 ฉบับ]

****What AI generated:****

ชุดข้อมูลทดสอบ 20 ข้อคำถาม-คำตอบ พร้อมระบุชื่อเอกสารอ้างอิงและมาตรากฎหมาย แต่ในตอนท้ายของบล็อก JSON มีการใส่แท็ก Markdown Citation ผิดมาด้วย เช่น `[cite: 1, 2, 3, 4, 5, 6]`

****What you changed or rejected:****

ตัดแท็ก cite และ Markdown backticks ส่วนเกินทิ้งทั้งหมด เพื่อรักษาความถูกต้องตามมาตรฐาน JSON Syntax ป้องกันการเกิด `JSONDecodeError` ขณะโหลดเข้าสู่ระบบประเมินผล

```

---

### Entry 4

**Prompt sent:**
```
Traceback (most recent call last):
 File 'scripts/run_eval.py', line 21, in
 'reference': item['reference_answer'],
KeyError: 'reference_answer'
ทำไม error ละเนี่ย
```

**What AI generated:**
ตรวจพบ Schema Mismatch ระหว่างคีย์ที่สคริปต์ `scripts/run_eval.py` เรียกใช้
(`reference_answer`) กับคีย์ที่อยู่ในไฟล์ `data/testset.json` ซึ่งสร้างไว้ในชื่อ
`ground_truth`

**What you changed or rejected:**
ปรับปรุงฟังก์ชันการดึงข้อมูลใน `scripts/run_eval.py` ให้มีความยืดหยุ่น (Robustness) โดยใช้
`ref = item.get("reference_answer") or item.get("ground_truth", "")` แทน
การเข้าถึงคีย์แบบเจาะจง เพื่อรองรับชุดข้อมูลทดสอบทั้งสองรูปแบบโดยไม่ต้องแปลงไฟล์ JSON ใหม่ทั้งหมด

```

```

---

### Entry 5

**Prompt sent:**
> `scripts/run_ablation.py, line 24 KeyError: 'relevant_docs'` ทำไมถึงรันไม่ผ่าน

**What AI generated:**
ชี้แจงว่าฟังก์ชันคำนวณ Information Retrieval Metrics (`recall_at_k`, `mrr`)
ต้องการ Ground Truth ในรูปของ List (`relevant_docs`) แต่ชุดข้อมูลทดสอบเก็บชื่อเอกสาร
ไว้ในคีย์ `source_doc` แบบ String เดียว

**What you changed or rejected:**
แก้ไข-loop ประเมินผลใน `scripts/run_ablation.py` ให้ตรวจสอบและดึง `target_docs =
item.get("relevant_docs") or [item.get("source_doc")]` พร้อมแปลงเป็น List

```

อัตโนมัติ รวมถึงตรวจสอบ `doc\_id` ในก้อน Chunk ที่ดึงขึ้นมาให้รองรับการ Match ชื่อไฟล์ที่มีหรือไม่มีนามสกุล `.pdf`/`.txt`

3. Decision Journal

#	Decision	Owner	Reason (in your own words)
---	---	---	---
1	**Corpus Chunking Strategy:** ตัด Chunk แบบ Token-aware ขนาด ~256 tokens (Overlap 32 tokens) Human ขนาด 256 tokens เหมาะกับความยาวเฉลี่ยของอนุมาตราและวรรคสำคัญในกฎหมายไทย ช่วยให้ Vector Representation มีความเฉพาะเจาะจงสูง (High Precision) ไม่เกิดปัญหา Dilution Effect อีกทั้งยังป้องกันไม่ให้ลำดับความยาว (Sequence Length) ล้นเกินขีดจำกัด 512 tokens ของ `multilingual-e5-base` เมื่อรวม prefix "passage: " และลดภาระความหน่วง (Latency) ในการส่ง Context เข้าสู่โมเดล Generator		
2	**Embedding Model:** เลือกใช้ `intfloat/multilingual-e5-base` AI-suggested, Human-approved รองรับภาษาไทยได้ดีผ่านการ Pre-train แบบ Cross-lingual สามารถรัน Inference บน CPU ได้อย่างมีประสิทธิภาพตามเงื่อนไขของโปรเจกต์โดยไม่ต้องพึ่ง GPU ขนาดใหญ่		
3	**Retrieval Architecture:** วางระบบเป็น 2-Stage (Hybrid Search BM25 + Dense ด้วย RRF Merging ตามด้วย Cross-Encoder Reranker) AI-suggested, Human-approved เอกสารกฎหมายมีทั้ง Keyword สำคัญเฉพาะเจาะจง (เช่น "มาตรา 4", "ผู้ค้าสินทรัพย์ดิจิทัล") ซึ่ง Dense เพียงอย่างเดียวจับได้ไม่ดีพอ การผสมผสาน BM25 เข้ามาช่วยกู้ Exact Keyword ได้แม่นยำขึ้น		
4	**Evaluation Metric Suite:** ใช้ทั้ง Generation Metrics (BERTScore, LLM-as-Judge) และ Retrieval Metrics (Recall@5, MRR) Human เพื่อแยกการประเมิน (Decouple) ประสิทธิภาพของส่วนค้นหาเอกสาร (Retriever) ออกจากส่วนสร้างข้อความ (LLM Generator) ช่วยให้เห็นจุดบกพร่องที่แท้จริงของ Pipeline		
5	**Environment & Text Encoding Policy:** กำหนด `setenv PYTHONUTF8=1` และจัดการ Windows Path Length Human Windows PowerShell มี Default Encoding เป็น CP874/CP1252 ซึ่งทำให้การ Print สาระภาษาไทยและเปิดไฟล์พัง การบังคับ UTF-8 และปรับโครงสร้าง Path จึงเป็นเงื่อนไขสำคัญต่อความเสถียรของระบบ		

4. Error Catch Log

Error 1

****What the AI said:****

เมื่อพบปัญหา Windows เปิดไฟล์ที่มี Path ยาวเกิน 260 ตัวอักษรไม่ได้ AI เสนอให้แก้โค้ดโดยการเติม Extended Path Prefix `\\?\\` เข้าไปที่ Path ของไฟล์โดยตรง เช่น
`\\?\D:\thai-legal-rag\data\pdfs\...` เพื่อสั่งให้ OS ปลอดภัย MAX_PATH

****Why it was wrong:****

ในทางทฤษฎี Win32 API รองรับ Prefix `\\?\\` แต่ในทางปฏิบัติ ฟังก์ชัน `fitz.open()` ของไลบรารี PyMuPDF พัฒนาด้วยแกน C/C++ (MuPDF engine) ซึ่งไม่ได้ส่งค่า Path ผ่าน Windows API ชั้นบนที่แปลง Prefix ดังกล่าว ตัว Engine ภายในจึงมองเห็น `\\?\\` เป็นอักขระใน Path จริง ทำให้โปรแกรมโยน Error `FileNotFoundError` ทันที

****How you fixed it:****

ปฏิเสธการแก้โค้ดด้วย Prefix และทำการแก้ไขที่สภาพแวดล้อมจริง โดยย้าย Root Folder ของโปรเจกต์มาไว้ที่ `D:\thai-legal-rag` เพื่อลดความยาว Base Path ลงเกือบ 80 ตัวอักษร และ Rename ไฟล์ PDF ภาษาไทยที่มีชื่อยาวเกินความจำเป็น ทำให้ระบบสามารถเปิดและสกัดข้อความได้ครบ 85/85 ไฟล์ (100%)

Error 2

****What the AI said:****

เมื่อสคริปต์สกัดข้อความ `extract\_corpus.py` แจ้งเตือนว่า "⚠ ไม่เจอคำว่า 'มาตรา' 2 ไฟล์" และแสดง Warning รายการไฟล์สภษา ขึ้นมา AI แนะนำให้ปรับ Regex หรืออาจต้องสั่งรัน OCR ใหม่ เพราะคาดว่าดึงข้อความไม่สำเร็จ

****Why it was wrong:****

AI ตวนสรุปว่าการไม่พบคำว่า "มาตรา" คือความล้มเหลวของการทำ Text Extraction แต่ในความเป็นจริงของบริบทกฎหมายไทย ทั้ง 2 ไฟล์ดังกล่าวคือ "ระเบียบสภาผู้แทนราษฎร" ซึ่งรูปแบบทางกฎหมายใช้คำว่า "ข้อ" (เช่น ข้อ 1, ข้อ 2) แทนคำว่า "มาตรา" ตัวข้อความในไฟล์ถูกสกัดออกมาอย่างสมบูรณ์แล้ว ไม่ได้เกิดข้อผิดพลาดในการอ่าน Text แต่อย่างใด

****How you fixed it:****

ตรวจสอบเนื้อหาไฟล์ `.txt` จริงในโฟลเดอร์ `data/documents` เพื่อยืนยันว่าข้อความครบถ้วน จากนั้นปรับเงื่อนไขในตัว Chunker ให้ตรวจจับทั้งคำว่า "มาตรา" สำหรับพระราชบัญญัติ/พระราชกำหนด และคำว่า "ข้อ" สำหรับระเบียบ/ข้อบังคับ เพื่อให้การแบ่ง Chunk ครอบคลุมเอกสารทุกประเภทใน Corpus

5. Contribution Map

Team Member	Human Contribution	AI Tools Used
[67070297]	รวบรวมเอกสารกฎหมายไทยเกี่ยวกับ [ภาษี, ธนาการ, การเงินและการลงทุน] (27 ฉบับ), ตรวจสอบความถูกต้องของ OCR, แก้ไขโครงสร้างไฟล์และ Path บนระบบปฏิบัติการ, ทดลองสร้าง Prototype RAG แบบ Full Pipeline, สร้างไฟล์ README.md, สร้างไฟล์ AI_AUDIT, สร้างไฟล์ json สำหรับ test จำนวน 20 ตัวอย่าง, แก้ Error, ทดลองสร้างเว็บไซต์ (Frontend) และ Deploy RAG (Backend) ขึ้น Web Server, ช่วยเพื่อนในกลุ่มทำสไลด์นำเสนอ	Claude Fable 5, Gemini Flash 3.8, Claude Opus 5, Claude Sonnet 5
[Member 2]	"ใส่งานที่ตัวเองทำ"	"ใส่ AI ที่ตัวเองใช้"
[Member 3]	"ใส่งานที่ตัวเองทำ"	"ใส่ AI ที่ตัวเองใช้"
[Member 4]	"ใส่งานที่ตัวเองทำ"	"ใส่ AI ที่ตัวเองใช้"

6. What Would Break?

> ตอบโดยไม่ใช่ AI

a) If you removed the reranker (or your second retrieval stage) from your pipeline, what specifically changes in output quality and why?

หากนำ Cross-Encoder Reranker (`bge-reranker`) ออกจากระบบ สิ่งที่จะลดลงทันทีคือ ****MRR (Mean Reciprocal Rank)**** และความกระชับของ Context ที่ส่งให้ LLM

****กลไกทางเทคนิค:**** ตัว Bi-Encoder (Dense Search) และ BM25 (Sparse Search) ทำงานแบบแยกวิเคราะห์ (Independent representation) โดย Bi-Encoder จะบีบอัดทั้งประโยคคำถามและเอกสารออกมาเป็น Vector เดียว ทำให้สูญเสียปฏิสัมพันธ์ระหว่างคำ (Token-to-token cross-attention) ส่งผลให้เอกสารที่มี Keyword ตรงกันแต่อยู่คนละบริบทอาจหลุดขึ้นมาอยู่ใน Top-3 ได้

เมื่อมี Cross-Encoder Reranker ทำหน้าที่ใน Stage 2 ตัว Reranker จะประมวลผลคู่ (Query, Document) พร้อมกันทั้งประโยคผ่าน Full Self-Attention ทำให้โมเดลเข้าใจเงื่อนไขความสัมพันธ์เชิงลึก และข้อยกเว้นของข้อกำหนดได้ดีกว่ามาก หากตัดออก Top-1 Document จะมีความแม่นยำน้อยลง ทำให้ LLM ได้รับ Context ที่มี Noise ปน และอาจนำไปสู่การสรุปคำตอบกฎหมายที่ผิดเพี้ยน

b) Why did you chunk your documents at ~512 tokens? What would happen to retrieval quality if you used 2x or 0.5x that size?

เราเลือกขนาด Chunk ที่ ~512 tokens เพราะสอดคล้องกับขนาดโดยเฉลี่ยของ "หนึ่งมาตราพร้อมวรรคขยายหรือบทยกเว้น" ในโครงสร้างกฎหมายไทย

****หากเพิ่มขนาดเป็น 2x (~1024 tokens):**** จะเกิดปัญหา Semantic Dilution (ความหมายเจือจาง) ตัว Embedding Model จะต้องบีบอัดข้อความ 2-3 มาตราที่อาจพูดคนละประเด็นลงในเวกเตอร์ 768 มิติอันเดิม ทำให้ Vector Representation มีค่าเฉลี่ยความหมายที่กว้างเกินไป (Loss of specificity) ส่งผลให้การค้นหาคำถามเจาะจงเฉพาะจุดทำได้แยกลง อีกทั้งยังเปลี่ยน Context Window ของ LLM โดยมีข้อความที่ไม่เกี่ยวข้องติดไปด้วย

****หากลดขนาดลงเหลือ 0.5x (~256 tokens):**** จะเกิดปัญหา Context Fragmentation (บริบทหักขาด) ตัวบทกฎหมายไทยมักบัญญัติหลักการไว้ในวรรคหนึ่ง และระบุข้อยกเว้นสำคัญไว้ในวรรคสองหรือวรรคท้าย หากตัดที่ 256 tokens บทยกเว้นจะถูกแยกขาดออกจากหลักการ ทำให้ระบบค้นหอาจหยิบเฉพาะท่อนหลักการมาตอบโดยไม่เห็นข้อยกเว้น ส่งผลให้ LLM ตอบข้อวินิจฉัยทางกฎหมายผิดพลาดอย่างร้ายแรง

c) Your ablation compares two system variants. Pick the result that surprised you most and explain the mechanism behind it – not just "variant A scored higher" but why technically.

ผลการทดลองที่น่าสนใจที่สุดคือ: ค่า ****Recall@5 เท่ากันที่ 0.5500**** ในทุก Variant แต่ค่า ****MRR เพิ่มขึ้นอย่างมีนัยสำคัญจาก 0.5167 (Dense) เป็น 0.5500 (Hybrid และ Hybrid+Reranker)****

****กลไกทางเทคนิคเบื้องหลัง:****

ทำไม Recall@5 ถึงเท่ากัน: ในชุดทดสอบ 20 ข้อ มีประมาณ 11 ข้อที่เนื้อหาเอกสารเกี่ยวข้องถูกต้องเข้ามาติด 1 ใน 5 อันดับแรกอยู่แล้วตั้งแต่รอบ Dense Search ส่วนอีก 9 ข้อที่เหลือที่ไม่ติด เกิดจากข้อจำกัดของการ Match ชื่อเอกสาร (String Mismatch ระหว่าง `doc\_id` กับ `source\_doc`) และคำถามบางข้อมีความเป็นนามธรรมสูงมากจนแม้แต่ BM25 ก็ไม่สามารถดึง Keyword เอกสารขึ้นมาติด Top-5 ได้ ทำให้ตัวเลขเพดานการครอบคลุม (Recall Pool) ใน 5 อันดับแรกไม่เปลี่ยนแปลง

ทำไม MRR จึงขยับสูงขึ้นอย่างชัดเจน: แม้จำนวนข้อที่ดึงเอกสารถูกต้องจะเท่าเดิม แต่ ****ตำแหน่งอันดับ (Rank Position)**** ของเอกสารที่ถูกต้องเปลี่ยนไปอย่างมีนัยสำคัญ สำหรับคำถามที่มีศัพท์เทคนิคเฉพาะ เช่น "การประกอบธุรกิจสินทรัพย์ดิจิทัล", "ตัวเงินค้ำประกัน", "เงินนอกงบประมาณ" ตัว Dense เวกเตอร์เพียงอย่างเดียวมักจัดเอกสารที่ตรงเป้าไปอยู่ในอันดับที่ 3 หรือ 4 แต่เมื่อมีกลไก BM25 เสริมแรงด้วย Reciprocal Rank Fusion (RRF) และตามด้วย Cross-Encoder Reranker ตัวระบบสามารถจับ Term Matching ของศัพท์เฉพาะเหล่านี้ได้ดีขึ้นมาก ส่งผลให้เอกสารที่เกี่ยวข้องถูกดันขึ้นมาอยู่ในอันดับที่ 1 และ 2 ค่าส่วนกลับของอันดับ ($1/\text{rank}$) จึงดีดตัวสูงขึ้นจาก 0.5167 เป็น 0.5500 อย่างชัดเจน ซึ่งส่งผลดีต่อตัว Generator (LLM) โดยตรงที่จะได้อ่านเอกสารสำคัญเป็นลำดับแรก

ไฟล์ [config.py](#)

```
"""Central configuration - ทุกค่าที่ต้อง 'defend' ตอน Q&A อยู่ที่นี่"""
import os
from pathlib import Path

# ----- Base Paths (Cross-Platform) -----
ROOT_DIR = Path(__file__).resolve().parent
DOCS_DIR = ROOT_DIR / "data" / "documents"
INDEX_DIR = ROOT_DIR / "artifacts"
TESTSET_PATH = ROOT_DIR / "data" / "testset.json"
RESULTS_PATH = ROOT_DIR / "artifacts" / "eval_results.json"

# ----- Chunking -----
# 256 tokens + overlap 48 สอดคล้องกับขนาดเฉลี่ย 1 มาตรากฎหมายไทย
CHUNK_SIZE = 256          # หน่วย: token ของ tokenizer
CHUNK_OVERLAP = 48

# ----- Embedding -----
EMBEDDING_MODEL = "intfloat/multilingual-e5-base"
E5_QUERY_PREFIX = "query: "
E5_PASSAGE_PREFIX = "passage: "

# ----- Retrieval & Reranker -----
TOP_K_RETRIEVE = 30      # ดึง candidate จาก Dense / BM25 ก่อนนำมารวมและ
Rerank (ปรับขึ้นจาก 20 เพื่อเพิ่ม Recall pool)
TOP_K_FINAL = 5          # จำนวน Context สูงสุดที่ส่งเข้า LLM
RRF_K = 60               # Smoothing constant มาตรฐาน RRF (Cormack et
al., 2009)
RERANKER_MODEL = "BAAI/bge-reranker-v2-m3"

# ----- Evaluation Metrics Cutoffs -----
EVAL_K_VALUES = [1, 3, 5] # สำหรับดูความแม่นยำในระดับ Rank ต่างๆ

# ----- LLM & Judge (Class Endpoint) -----
LLM_BASE_URL = "https://llm.nat-d.uk/v1"
# ดึงจาก Environment Variable ก่อน ถ้าไม่มีให้ fallback ไปที่คีย์เดิม
LLM_API_KEY = os.getenv("LLM_API_KEY", "sk-qLAWKH1pgFQIxG-2fRCGpg")
```

```
# ใช้โมเดล gemma-4-E4B-it ทั้งคู่ตาม Resource ที่ได้รับจากคลาส
LLM_MODEL = "gemma-4-E4B-it"
JUDGE_MODEL = "gemma-4-E4B-it"
JUDGE_TEMPERATURE = 0.0

# ข้อความมาตรฐานเมื่อระบบปฏิเสธการตอบเพื่อลด Hallucination
ABSTAIN_TEXT = "ไม่พบข้อมูลในเอกสารที่มี"
```

ไฟล์ [demo.py](#)

```
import sys
sys.path.insert(0, ".")

from src.pipeline import RAGPipeline

# สำหรับ Live Demo: หากต้องการให้ Rerank ไวชั่น สามารถใช้ mode="hybrid" หรือคง
"hybrid_rerank" ไว้ได้
pipeline = RAGPipeline(mode="hybrid_rerank")

print("Thai Legal RAG - พิมพ์คำถาม (พิมพ์ 'exit' เพื่อออก)\n")

while True:
    try:
        q = input(" ? คำถาม: ").strip()
        if not q:
            continue
        if q.lower() in ("exit", "quit"):
            break

        print("\n💡 คำตอบ: ")
        # ดึง Contexts และ Iterator ของ Stream
        # ส่ง retrieve_k=12 หรือ 15 เพื่อลดภาระ CPU ให้ตอบไวขึ้นตอนเดโมสด
        contexts, stream = pipeline.answer_stream(q, retrieve_k=12)

        # พิมพ์ตัวอักษรออกมาทีละ Token ทันที
        for token in stream:
            print(token, end="", flush=True)
        print("\n")
```

```

print("📄 แหล่งอ้างอิง:")
for c in contexts:
    score = c.get("rerank_score", c.get("score", 0.0))
    doc_name = c.get("doc_id") or c.get("file")
    print(f" - {doc_name} (score: {score:.4f})")
print()

except KeyboardInterrupt:
    print("\nออกจากโปรแกรม")
    break

```

ไฟล์ [README.md](#)

🇹🇭 Thai Legal RAG Q&A System

ระบบถาม-ตอบเอกสารกฎหมายไทยด้วยสถาปัตยกรรม Retrieval-Augmented Generation (RAG) ผสาน Hybrid Search (Dense Vector + BM25) ร่วมกับ Cross-Encoder Reranker พร้อมระบบ OCR สำหรับเอกสาร PDF ที่ Text Layer เสียหาย

📄 สารบัญ

- [1. การเตรียมระบบและติดตั้ง OCR Engine] (#1-การเตรียมระบบและติดตั้ง-ocr-engine-system-dependencies)
- [2. การติดตั้ง Dependencies และสภาพแวดล้อม] (#2-การติดตั้ง-dependencies-และสภาพแวดล้อม-python-environment)
- [3. การตั้งค่าระบบ] (#3-การตั้งค่าระบบ-configuration)
- [4. ลำดับการรัน Pipeline แบบทีละขั้นตอน] (#4-ลำดับการรัน-pipeline-แบบทีละขั้นตอน-step-by-step-execution)
- [5. การรัน Pipeline อัตโนมัติด้วยคำสั่งเดียว] (#5-การรัน-pipeline-อัตโนมัติด้วยคำสั่งเดียว-one-click-pipeline)
- [6. ผลลัพธ์และโครงสร้าง Artifacts] (#6-ผลลัพธ์และโครงสร้าง-artifacts)

1. การเตรียมระบบและติดตั้ง OCR Engine (System Dependencies)

โปรเจกต์นี้มีระบบ ****Fallback อัปเดตอัตโนมัติไปยัง Tesseract OCR**** สำหรับเอกสาร PDF กฎหมายที่ไม่มี Text Layer หรือข้อความเสียหาย จำเป็นต้องติดตั้ง Tesseract พร้อมข้อมูลภาษาไทย (`tha`) ตามระบบปฏิบัติการดังนี้

🪟 Windows

1. ดาวน์โหลดตัวติดตั้ง (`.exe`) สำหรับ 64-bit จาก [UB-Mannheim/tesseract/wiki] (<https://github.com/UB-Mannheim/tesseract/wiki>)
2. เปิดตัวติดตั้ง กด ****Next**** จนถึงหน้าต่าง ****Choose Components****
3. คลิกหัวข้อ ****Additional script data**** และ ****Additional language data**** แล้วติ๊กเลือก ****Thai****
4. ตรวจสอบให้แน่ใจว่าติดตั้งไว้ที่ Path เริ่มต้น (`C:\Program Files\Tesseract-OCR\tesseract.exe`)

🐧 Ubuntu / Debian / WSL

```
```bash
sudo apt-get update
sudo apt-get install -y tesseract-ocr tesseract-ocr-tha
```
```

🍏 macOS (Apple Silicon / Intel)

```
```bash
brew install tesseract tesseract-lang
```
```

2. การติดตั้ง Dependencies และสภาพแวดล้อม (Python Environment)

แนะนำให้ใช้ Python 3.10+ โดยแยกการตั้งค่าสภาพแวดล้อมตาม Terminal ที่ใช้งาน

🪟 Windows (Command Prompt - CMD)

```
```dos
:: บังคับใช้ UTF-8 เพื่อป้องกันข้อผิดพลาดจากภาษาไทยและสระลอย
```

```
chcp 65001
set PYTHONUTF8=1

:: สร้างและเปิดใช้งาน Virtual Environment
py -m venv .venv
call .venv\Scripts\activate.bat

:: อัปเดต pip และติดตั้งไลบรารี
py -m pip install --upgrade pip
py -m pip install -r requirements.txt
```
```

🪟 Windows (PowerShell)

```
```powershell
บังคับใช้ UTF-8
$env:PYTHONUTF8=1

สร้างและเปิดใช้งาน Virtual Environment
py -m venv .venv
.\.venv\Scripts\Activate.ps1

อัปเดต pip และติดตั้งไลบรารี
py -m pip install --upgrade pip
py -m pip install -r requirements.txt
```
```

🐧 🍏 Linux & macOS (Bash / Zsh)

```
```bash
บังคับใช้ UTF-8
export PYTHONUTF8=1
export PYTHONIOENCODING=utf-8

สร้างและเปิดใช้งาน Virtual Environment
python3 -m venv .venv
source .venv/bin/activate

อัปเดต pip และติดตั้งไลบรารี
pip install --upgrade pip
```
```

```
pip install -r requirements.txt
```
```

---

### ## 3. การตั้งค่าระบบ (Configuration)

ตรวจสอบไฟล์ `config.py` เพื่อระบุค่า Endpoint และ Key สำหรับโมเดลภาษา:

```
```python
LLM_BASE_URL = "https://llm.nat-d.uk/v1" # หรือคลาสเรียน Endpoint / OpenAI
API
LLM_API_KEY = "YOUR_API_KEY"
LLM_MODEL = "gemma-4-E4B-it" # โมเดล Generator หลัก
JUDGE_MODEL = "gemma-4-E4B-it" # โมเดล LLM-as-Judge
```
```

> ⚠️ **\*\*หมายเหตุ\*\*:** หลีกเลี่ยงการ Commit ไฟล์ที่มี API Key ขึ้น Git โดยเด็ดขาด

---

### ## 4. ลำดับการรัน Pipeline แบบทีละขั้นตอน (Step-by-Step Execution)

#### ### ขั้นตอนที่ 1 – สกัดข้อความจาก PDF (Data Ingestion)

แปลงไฟล์ PDF ข้อกฎหมายใน `data/pdfs/` ให้เป็นข้อความที่ผ่านการ Clean ใน  
`data/documents/` พร้อมระบบ OCR สำรอง

```
```bash
# Windows (CMD / PowerShell)
py src/extract_corpus.py

# Linux / macOS
python3 src/extract_corpus.py
```
```

#### ### ขั้นตอนที่ 2 – แบ่งท่อนข้อความและสร้าง Index (Build Indexes)

ตัดข้อความแบบ Token-aware Chunks (~256 tokens) และสร้าง Dense Index  
(multilingual-e5-base) ควบคู่กับ Sparse Index (BM25)

```
```bash
# Windows (CMD / PowerShell)
py scripts/build_index.py

# Linux / macOS
python3 scripts/build_index.py
```
```

### ### ขั้นตอนที่ 3 – ตรวจสอบความพร้อมของระบบ (Smoke Test)

ตรวจสอบความสอดคล้องของ Config, Metrics, Testset, Corpus และ Retriever ก่อนเริ่มทดลองจริง

```
```bash
# Windows (CMD / PowerShell)
py scripts/smoke_test.py

# Linux / macOS
python3 scripts/smoke_test.py
```
```

ต้องผ่านการทดสอบขั้นเครื่องหมาย  ครบทุกหัวข้อ

### ### ขั้นตอนที่ 4 – รันการทดลองเปรียบเทียบ Retrieval (Ablation Study)

วัดประสิทธิภาพเปรียบเทียบ 4 Variants (dense, bm25, hybrid, hybrid\_rerank) บน Metrics: Recall@k, MRR@k, nDCG@k, Latency และค่าสถิติ Bootstrap CI

```
```bash
# Windows (CMD / PowerShell)
py scripts/run_ablation.py

# Linux / macOS
python3 scripts/run_ablation.py
```
```

### ### ขั้นตอนที่ 5 – ประเมินผลระบบถาม-ตอบแบบครบวงจร (Full Evaluation)

รัน End-to-End Pipeline เพื่อวัด Semantic Similarity (BERTScore  
xlm-roberta-large), LLM-as-Judge (Faithfulness & Relevance), อัตราการปฏิเสธคำ  
ตอบที่อยู่นอกขอบเขต (Abstention Rate) และสเกดเคสล้มเหลวอัตโนมัติ

```
```bash
# Windows (CMD / PowerShell)
py scripts/run_eval.py

# Linux / macOS
python3 scripts/run_eval.py
```
```

### ### ขั้นตอนที่ 6 – ทดสอบระบบถาม-ตอบสด (Interactive Live Demo)

รันหน้าจอโต้ตอบสดแบบ Streaming (พิมพ์ทีละ Token) พร้อมแสดงเอกสารและมาตราอ้างอิง

```
```bash
# Windows (CMD / PowerShell)
py demo.py

# Linux / macOS
python3 demo.py
```
```

(พิมพ์คำถามกฎหมายที่ต้องการ หรือพิมพ์ `exit` เพื่อออกจากระบบ)

---

## ## 5. การรัน Pipeline อัตโนมัติด้วยคำสั่งเดียว (One-Click Pipeline)

### ### 🖥️ Windows (PowerShell)

```
```powershell
.\run_pipeline.ps1
```
```

### ### 🐧 🍏 Linux / macOS (Bash)

```
```bash
chmod +x run_pipeline.sh
```

```
./run_pipeline.sh
```

```
```
```

```

```

## ## 6. ผลลัพธ์และโครงสร้าง Artifacts

หลังการรัน Pipeline ครบถ้วน ข้อมูลผลลัพธ์ทั้งหมดจะถูกจัดเก็บไว้ในโฟลเดอร์ `artifacts/` ดังนี้:

```
ไฟล์ Artifact	รายละเอียดของข้อมูล	การนำไปใช้งาน
`dense.faiss`	Vector Index สำหรับการค้นหาเชิงความหมาย (Bi-Encoder)	ใช้งาน
ใน Retriever Mode Dense / Hybrid		
`bm25.pkl`	Sparse Index สำหรับการค้นหาด้วยคีย์เวิร์ดเฉพาะทางกฎหมาย	ใช้งานใน
Retriever Mode BM25 / Hybrid		
`chunks.json`	ก้อนข้อความกฎหมายทั้งหมดหลังการทำ Token-aware Chunking	ใช้
อ้างอิงและประกอบ Context ส่งเข้า LLM		
`ablation_results.json`	ผลการทดลองเปรียบเทียบทั้ง 4 Retrieval Variants เชิง	
ตัวเลข	ใช้ตรวจสอบสถิติละเอียด (Bootstrap CI, Latency)	
`ablation_table.md`	ตารางสรุปผล Retrieval Metrics ในรูปแบบ Markdown	
สำหรับนำไปวางใน Presentation Slides (หน้า 6-7)		
`eval_results.json`	ผลการประเมิน BERTScore, LLM Judge และ Abstention	
Rate	สำหรับนำไปวางใน Presentation Slides (หน้า 8)	
`failure_cases.json`	บันทึกเคสที่ระบบปฏิเสธผิดพลาดหรือดึงเอกสารไม่สมบูรณ์	สำหรับนำ
ไปใช้อภิปรายใน Failure Analysis (หน้า 9) |
```

---

ไฟล์ requirements.txt

sentence-transformers>=3.0

transformers>=4.40

torch

rank-bm25

pythainlp>=5.0

numpy

faiss-cpu

bert-score

openai

tqdm

pymupdf>=1.24  
pytesseract  
pillow

ไฟล์ run\_pipeline.ps1

```
#
=====
====
Pipeline Execution Script for Windows (PowerShell)
#
=====
====

$ErrorActionPreference = "Stop"

บังคับใช้ Encoding UTF-8
$env:PYTHONUTF8 = 1
$OutputEncoding = [System.Text.Encoding]::UTF8

Write-Host "=====
-ForegroundColor Cyan
Write-Host " Starting Full Legal-RAG Pipeline (Windows)" -ForegroundColor
Cyan
Write-Host "=====
-ForegroundColor Cyan

#

ขั้นตอนที่ 0: ตรวจสอบ Tesseract OCR
#

Write-Host "`n>>> [0/5] Checking Tesseract OCR..." -ForegroundColor Yellow
$tessPath = "C:\Program Files\Tesseract-OCR\tesseract.exe"
if (Test-Path $tessPath) {
 Write-Host "Tesseract OCR found at $tessPath" -ForegroundColor Green
} else {
```

```
Write-Warning "Tesseract OCR not found at default path. Make sure it
is installed if OCR fallback is needed."
}

#

ขั้นตอนที่ 1: Virtual Environment & Dependencies
#

Write-Host "`n>>> [1/5] Setting up Python Environment..." -ForegroundColor
Yellow

if (-not (Test-Path ".venv")) {
 Write-Host "Creating virtual environment (.venv)..."
 py -m venv .venv
}

Write-Host "Activating virtual environment..."
& .\.venv\Scripts\Activate.ps1

Write-Host "Upgrading pip and installing requirements..."
py -m pip install --upgrade pip
py -m pip install -r requirements.txt

#

ขั้นตอนที่ 2: Data Extraction (PDF -> Clean Text)
#

Write-Host "`n>>> [2/5] Extracting & Cleaning Text from Legal PDFs..."
-ForegroundColor Yellow
py src/extract_corpus.py

#


```

```
ขั้นตอนที่ 3: Chunking & Indexing (FAISS + BM25)
#

Write-Host "`n>>> [3/5] Building Vector & BM25 Indexes..."
-ForegroundColor Yellow
py scripts/build_index.py

#

ขั้นตอนที่ 4: Full Evaluation (BERTScore & LLM Judge)
#

Write-Host "`n>>> [4/5] Running Model Evaluation..." -ForegroundColor
Yellow
py scripts/run_eval.py

#

ขั้นตอนที่ 5: Retrieval Ablation Study (Dense vs Hybrid vs Rerank)
#

Write-Host "`n>>> [5/5] Running Ablation Experiments..." -ForegroundColor
Yellow
py scripts/run_ablation.py

Write-Host "`n======"
-ForegroundColor Green
Write-Host " All Pipeline Stages Completed Successfully!" -ForegroundColor
Green
Write-Host " Results saved in artifacts/" -ForegroundColor Green
Write-Host "======"
-ForegroundColor Green
```

---

## ไฟล์ run\_pipeline.sh

```
#!/usr/bin/env bash
#
=====
====
Pipeline Execution Script for thai-legal-rag
Supports: macOS & Linux / Ubuntu / WSL
#
=====
=====

set -e # หยุดการทำงานทันทีหากคำสั่งใดเกิด error

กำหนด Encoding เป็น UTF-8 ป้องกันสระภาษาไทยเพี้ยน
export PYTHONUTF8=1
export PYTHONIOENCODING=utf-8

echo "=====
echo " Starting Full Legal-RAG Pipeline"
echo "=====

#

ขั้นตอนที่ 0: ติดตั้ง/ตรวจสอบ Tesseract OCR ภาษาไทย
#

echo -e "\n>>> [0/5] Checking OCR System Dependencies..."

if [["$OSTYPE" == "darwin"*]]; then
 # macOS
 if ! command -v tesseract &> /dev/null; then
 echo "Tesseract not found. Installing via Homebrew..."
 brew install tesseract tesseract-lang
 else
 echo "Tesseract found: $(which tesseract)"
 fi
elif [["$OSTYPE" == "linux-gnu"*]]; then
```

```
Linux / Ubuntu / Debian / WSL
if ! command -v tesseract &> /dev/null; then
 echo "Tesseract not found. Installing via apt..."
 sudo apt-get update
 sudo apt-get install -y tesseract-ocr tesseract-ocr-tha
else
 echo "Tesseract found: $(which tesseract)"
fi
fi

#

ขั้นตอนที่ 1: ตั้งค่า Python Virtual Environment & Dependencies
#

echo -e "\n>>> [1/5] Setting up Python Environment..."

if [! -d ".venv"]; then
 echo "Creating virtual environment (.venv)..."
 python3 -m venv .venv
fi

echo "Activating virtual environment..."
source .venv/bin/activate

echo "Upgrading pip and installing requirements..."
pip install --upgrade pip
pip install -r requirements.txt

#

ขั้นตอนที่ 2: Data Extraction (PDF -> Clean Text)
#

echo -e "\n>>> [2/5] Extracting & Cleaning Text from Legal PDFs..."
python3 src/extract_corpus.py
```

```
#

ขั้นตอนที่ 3: Chunking & Indexing (FAISS + BM25)
#

echo -e "\n>>> [3/5] Building Vector & BM25 Indexes..."
python3 scripts/build_index.py

#

ขั้นตอนที่ 4: Full Evaluation (BERTScore & LLM Judge)
#

echo -e "\n>>> [4/5] Running Model Evaluation..."
python3 scripts/run_eval.py

#

ขั้นตอนที่ 5: Retrieval Ablation Study (Dense vs Hybrid vs Rerank)
#

echo -e "\n>>> [5/5] Running Ablation Experiments..."
python3 scripts/run_ablation.py

echo -e "\n=====
echo " All Pipeline Stages Completed Successfully!"
echo " Results saved in artifacts/"
echo "====="
```